

Visione Artificiale

Raffaella Lanza

Video

Video representation

✓ **Video = 2D + Time**

A video is a sequence of images
4D tensor: $T \times 3 \times H \times W$
(or $3 \times T \times H \times W$)



Example task: Video Classification



Images: Recognize **objects**



Dog
Cat
Fish
Truck



Videos: Recognize **actions**



Swimming
Running
Jumping
Eating
Standing

Problem: Videos are big!



Input video:

$T \times 3 \times H \times W$

- ✓ Videos are ~30 frames per second (fps)
- ✓ Size of uncompressed video
- ✓ (3 bytes per pixel):
 - SD (640 x 480): **~1.5 GB per minute**
 - HD (1920 x 1080): **~10 GB per minute**
- ✓ **Solution:** Train on short **clips**: low fps and low spatial resolution e.g. $T = 16$, $H=W=112$ (3.2 seconds at 5 fps, 588 KB)

Training on Clips

- ✓ **Train on short clips:** low fps and low spatial resolution
- ✓ e.g. $T = 16, H = W = 112$ (3.2 seconds at 5 fps, 588 KB)

- ✓ **Raw video:** Long, high FPS



- ✓ **Training:** Train model to classify short clips with low FPS



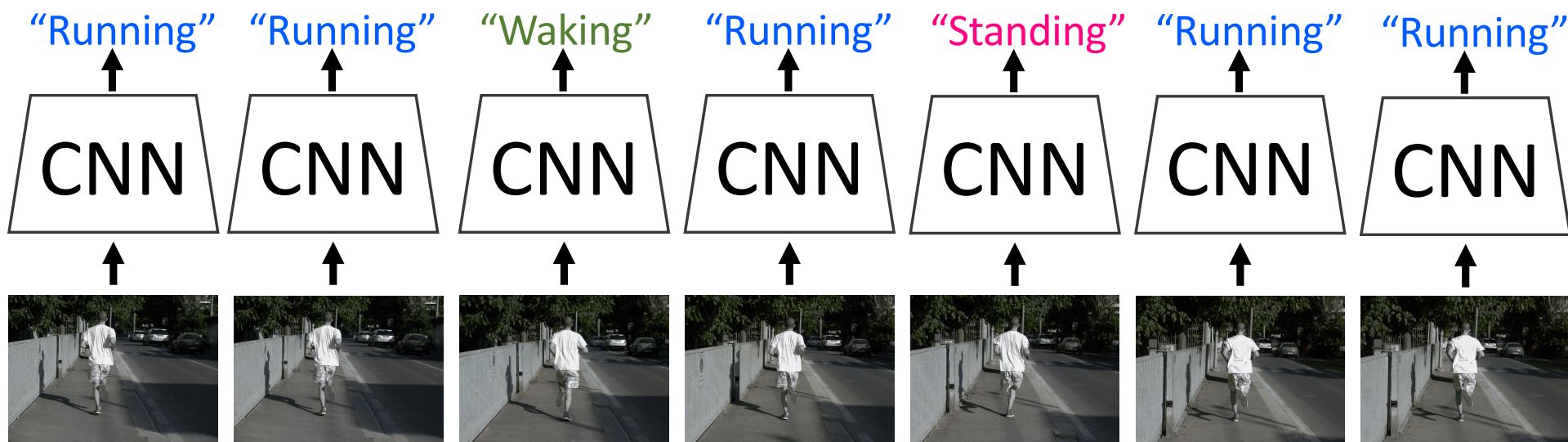
- ✓ **Testing:** Run model on different clips, average predictions



Video classification

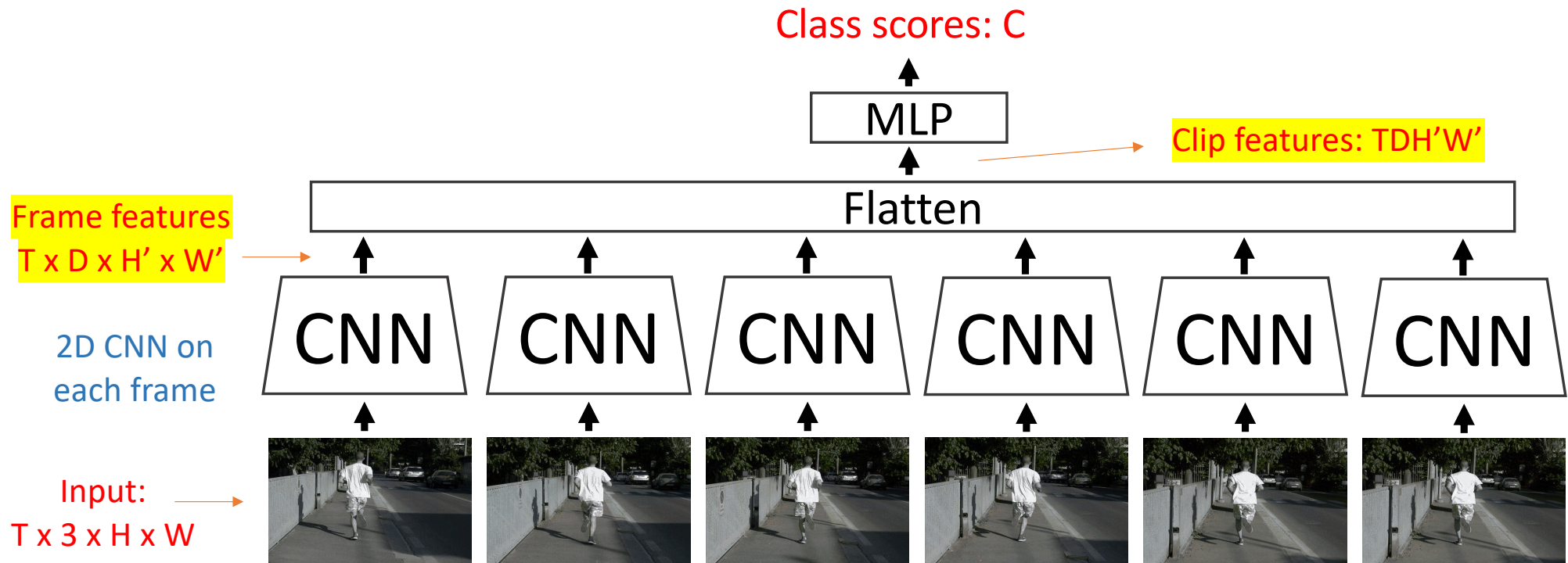
Video Classification: Single-Frame CNN

- ✓ **Simple idea:** train normal **2D CNN** to classify video **frames independently**
- ✓ **Average predicted** probs at test-time
- ✓ Often a **very strong baseline** for video classification



Video Classification: Late Fusion (with FC layers)

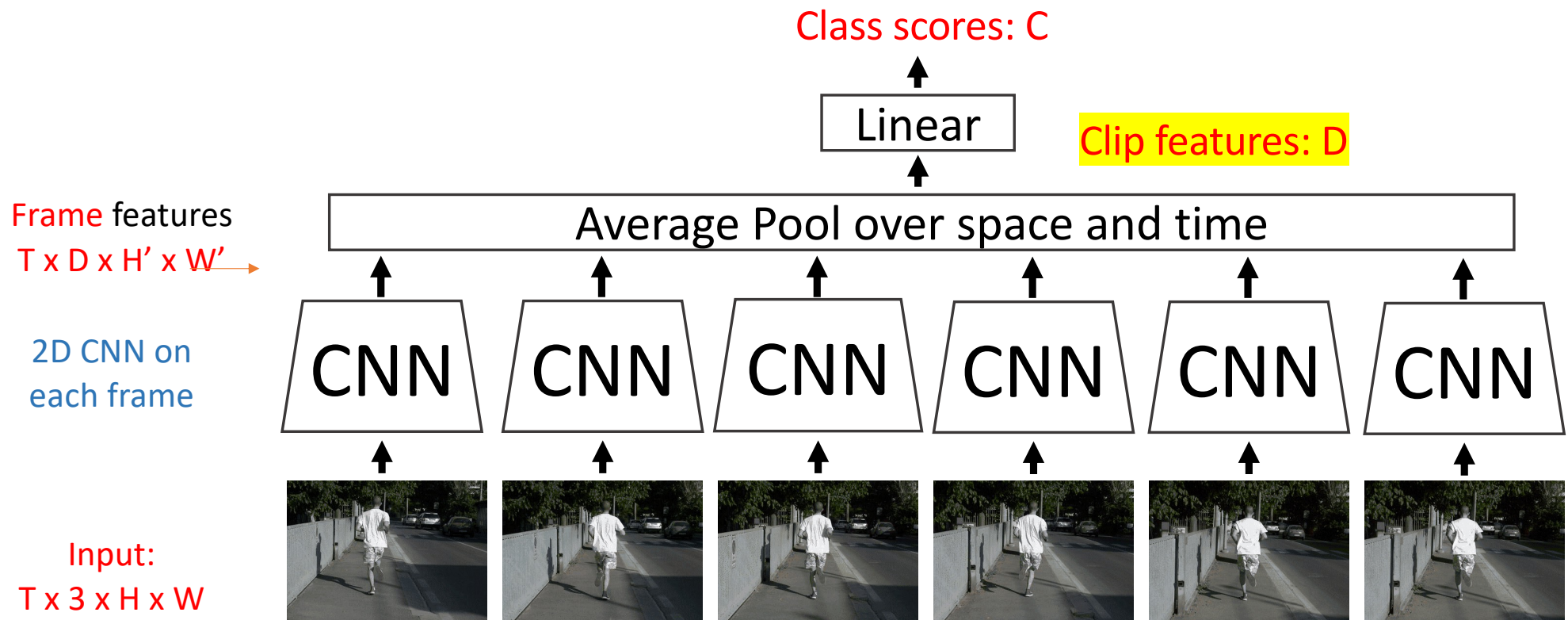
- ✓ **Intuition:** Get high-level appearance of each frame, and combine them
- ✓ Run **2D CNN** on **each frame**, **concatenate features** and feed to **MLP**



Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

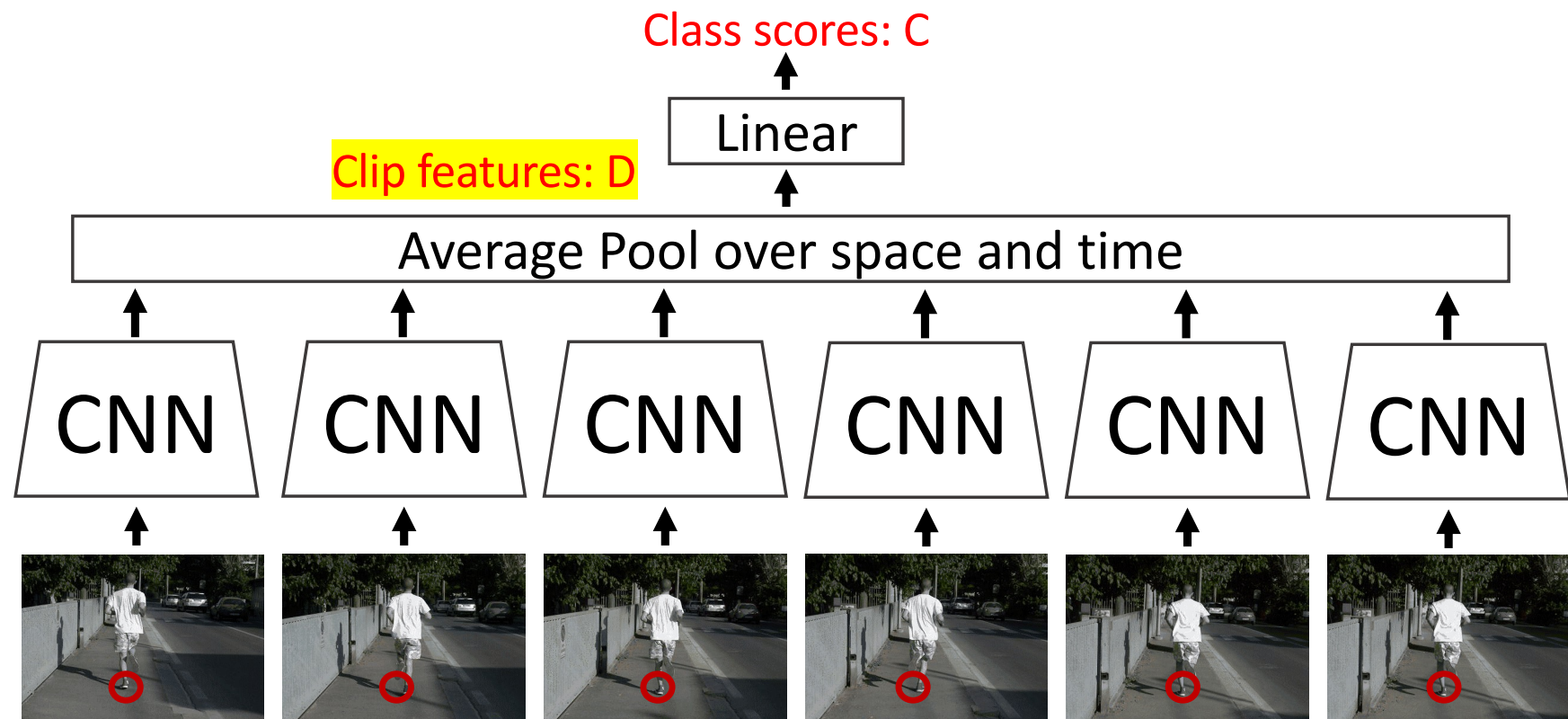
Video Classification: Late Fusion (with pooling)

- ✓ **Intuition:** Get high-level appearance of each frame, and combine them
- ✓ Run **2D CNN** on **each frame**, **concatenate features** and feed to **Linear**



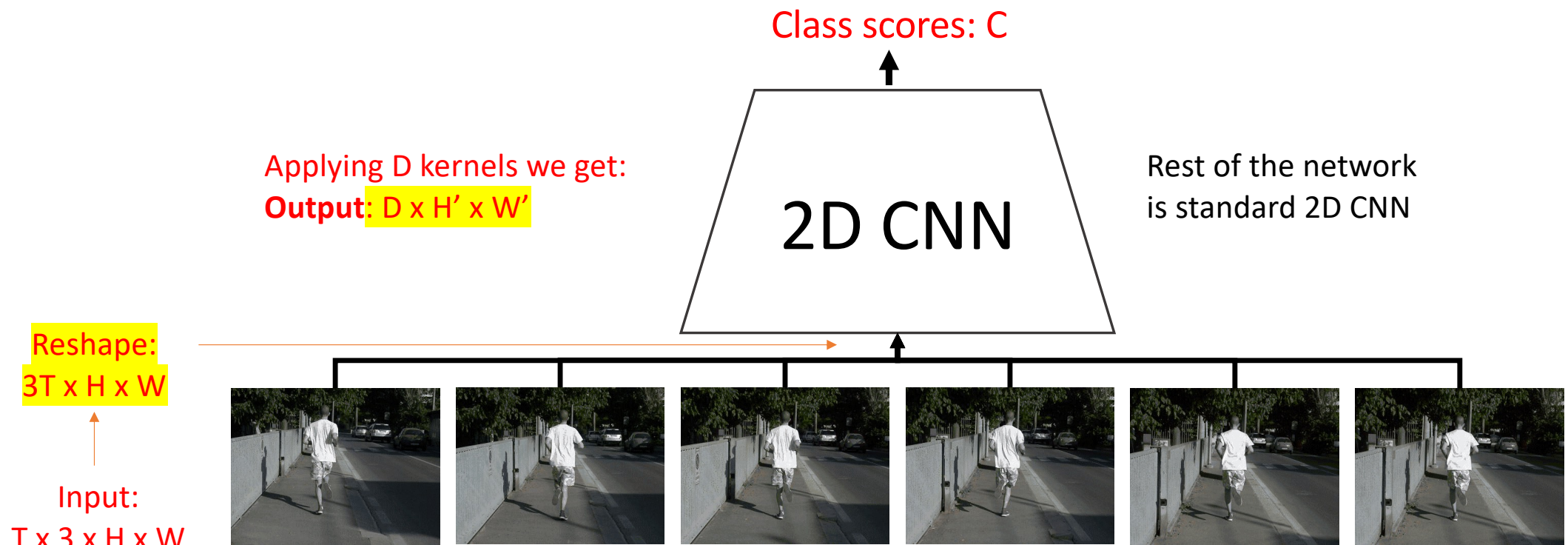
Video Classification: Late Fusion (with pooling)

Problem: Hard to compare low-level motion between frames!



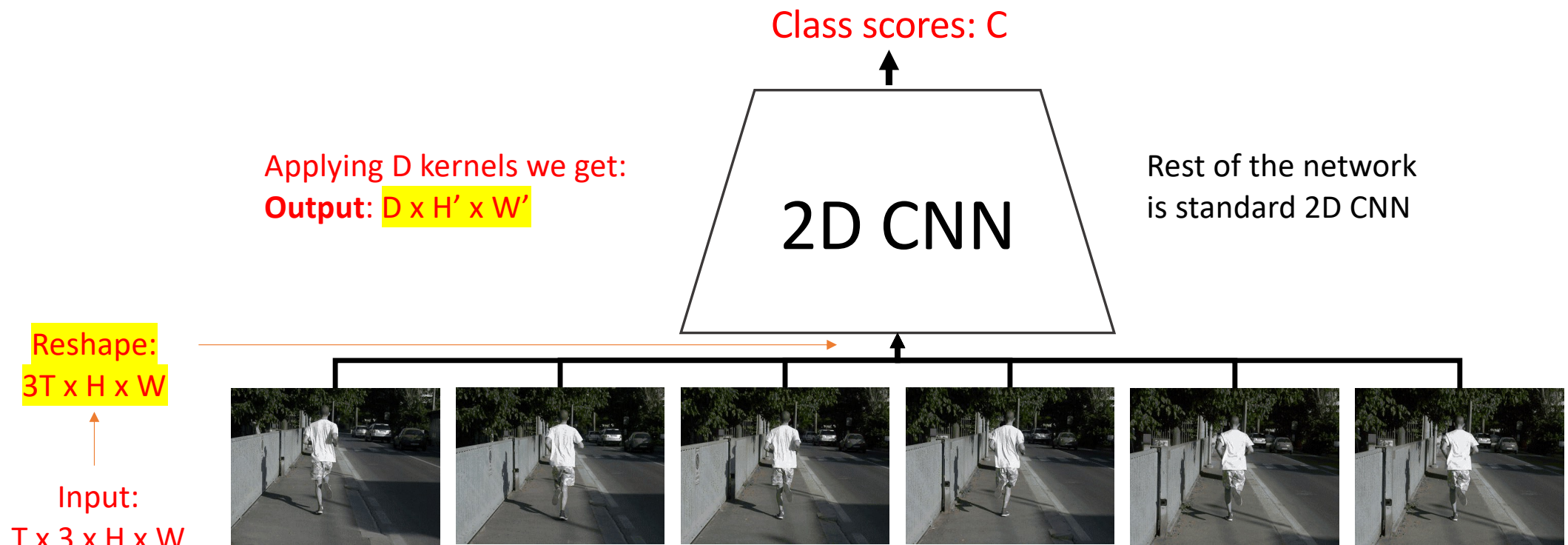
Video Classification: Early Fusion

Intuition: Compare frames with very first conv layer, after that normal 2D CNN



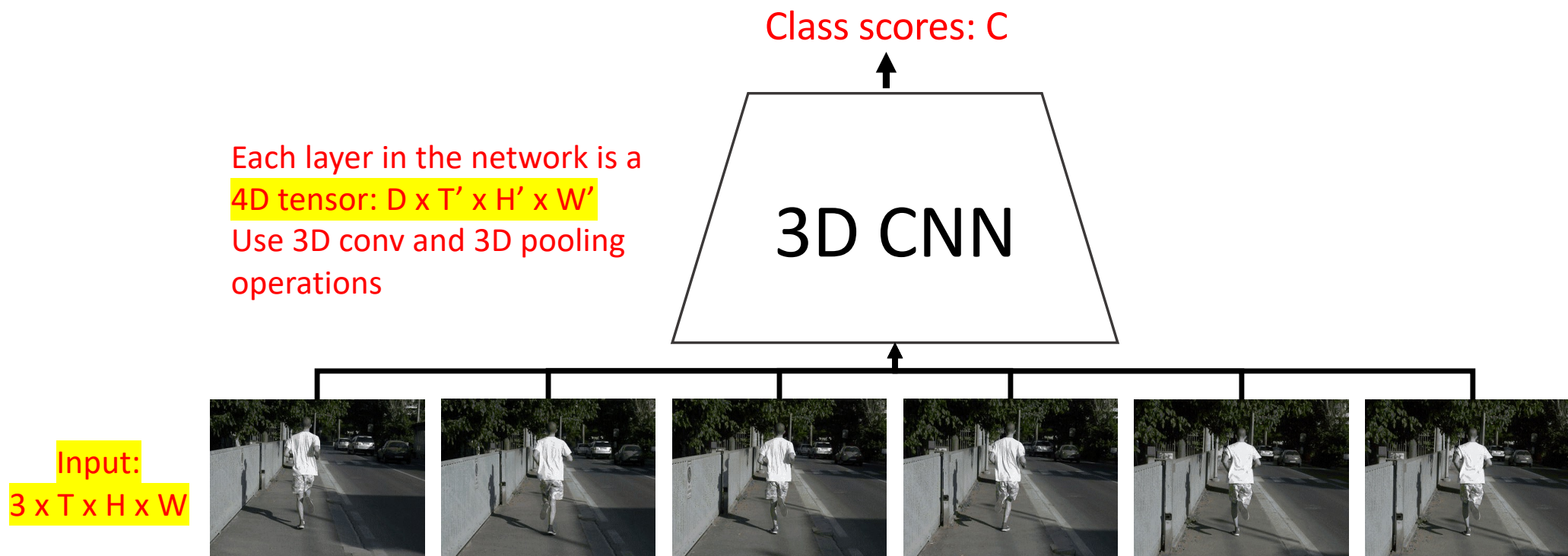
Video Classification: Early Fusion

Problem: One layer of temporal processing may not be enough



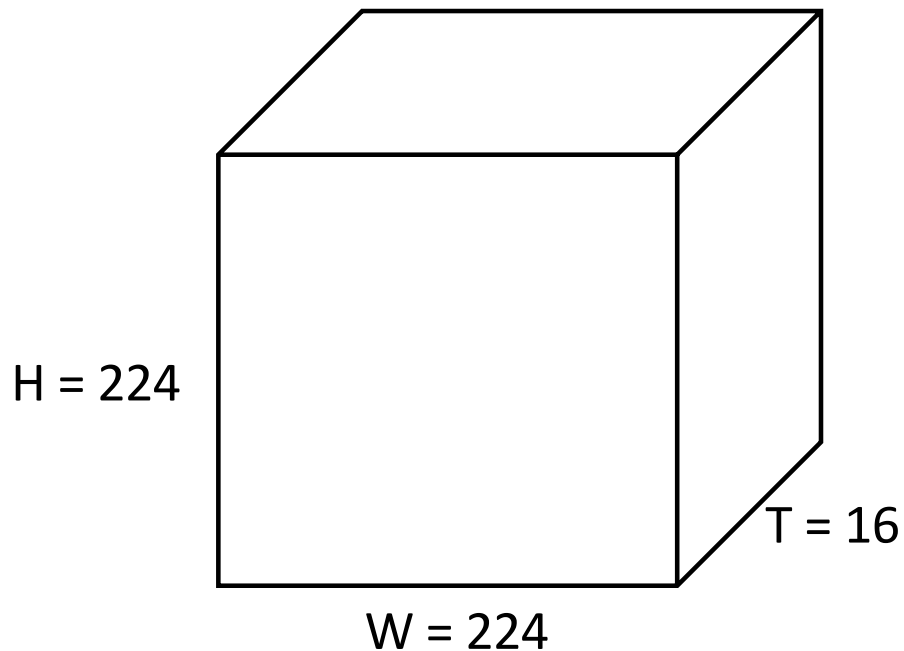
Video Classification: 3D CNN

- ✓ **Intuition:** Use **3D versions of convolution** and **pooling** to **slowly fuse** temporal information over the course of the network

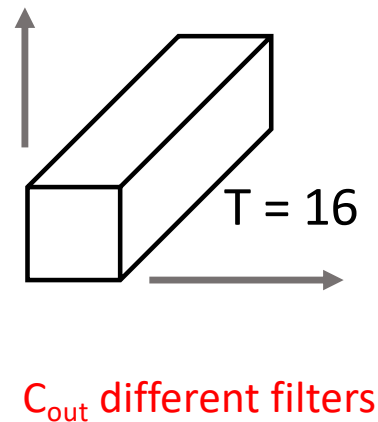


2D Conv (Early Fusion) vs 3D Conv (3D CNN)

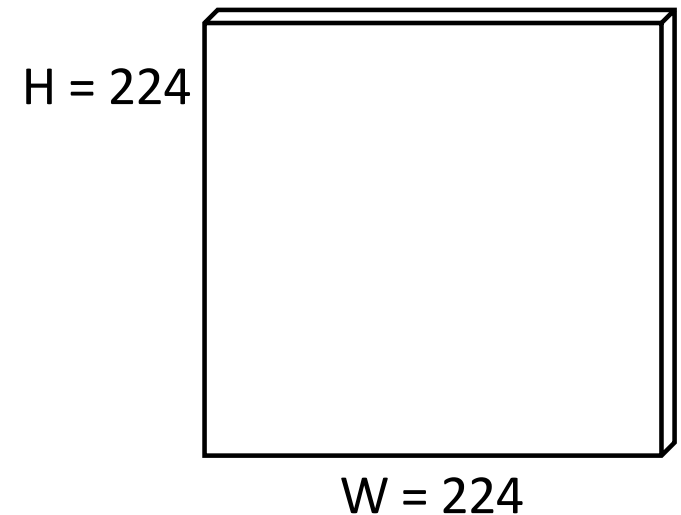
Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



Weight:
 $C_{out} \times C_{in} \times T \times 3 \times 3$
Slide over x and y

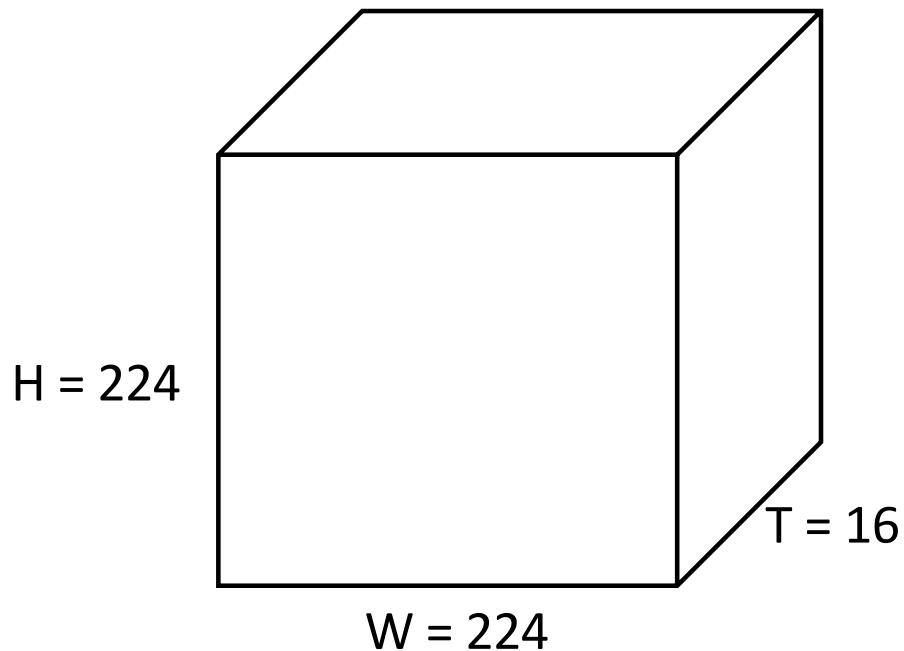


Output:
 $C_{out} \times H \times W$
2D grid with C_{out} -dim
feat at each point



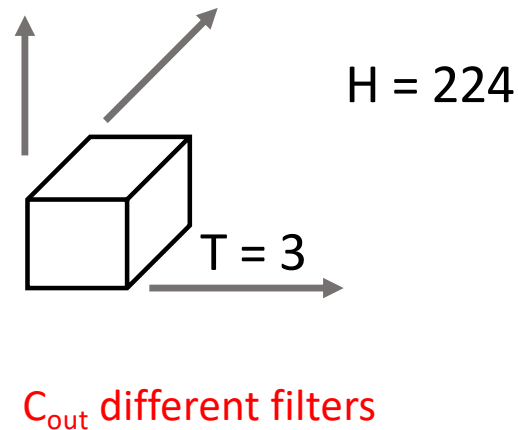
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: $C_{in} \times T \times H \times W$
(3D grid with C_{in} -dim
feat at each point)



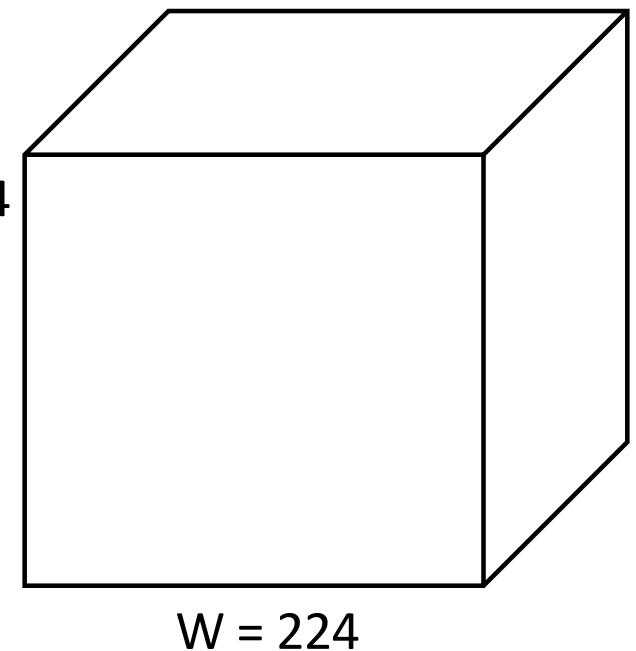
Weight:

$C_{out} \times C_{in} \times \mathbf{3} \times 3 \times 3$
Slide over x and y



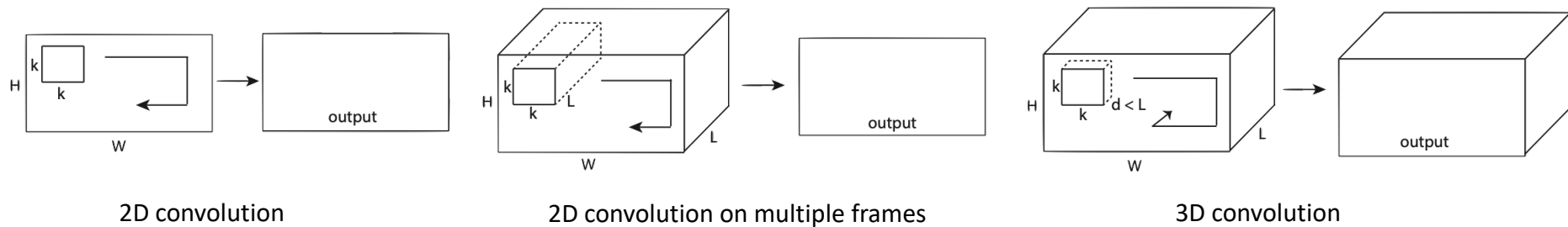
Output:

$C_{out} \times T \times H \times W$
3D grid with C_{out} -dim
feat at each point

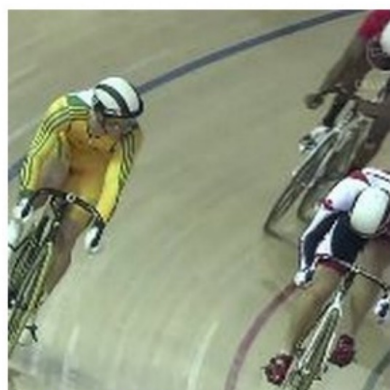


Rationale

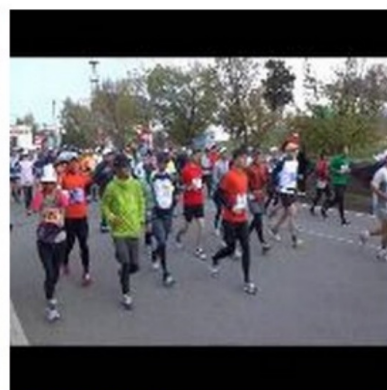
- ✓ In **3D ConvNets**, **convolution** and **pooling** operations are performed **spatio-temporally** while in **2D ConvNets** they are done only **spatially**
- ✓ **2D convolution** applied on an image will output an **image**, 2D convolution applied on **multiple images** (treating them as different channels) also results in an image
- ✓ **2D ConvNets loose temporal** information of the input signal right after every convolution operation
- ✓ Only **3D convolution preserves the temporal information** of the input signals resulting in an **output volume**



Example Video Dataset: Sports-1M



track cycling
cycling
track cycling
road bicycle racing
marathon
ultramarathon



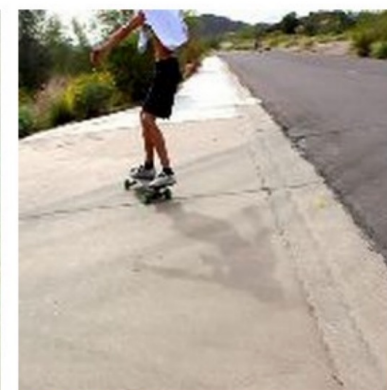
ultramarathon
ultramarathon
half marathon
running
marathon
inline speed skating



heptathlon
heptathlon
decathlon
hurdles
pentathlon
sprint (running)



bikejoring
mushing
bikejoring
harness racing
skijoring
carting



longboarding
longboarding
aggressive inline skating
freestyle scootering
freeboard (skateboard)
sandboarding

1 million YouTube videos
annotated with labels for
487 different types of sports

Ground Truth

Correct prediction

Incorrect prediction

C3D: the VGG of 3D CNNs

- ✓ **3D CNN** that uses all **3x3x3 conv** and **2x2x2 pooling**
- ✓ (except Pool1 which is 1x2x2)
- ✓ Released model **pretrained** on **Sports-1M**: Many people used this as a video feature extractor

Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

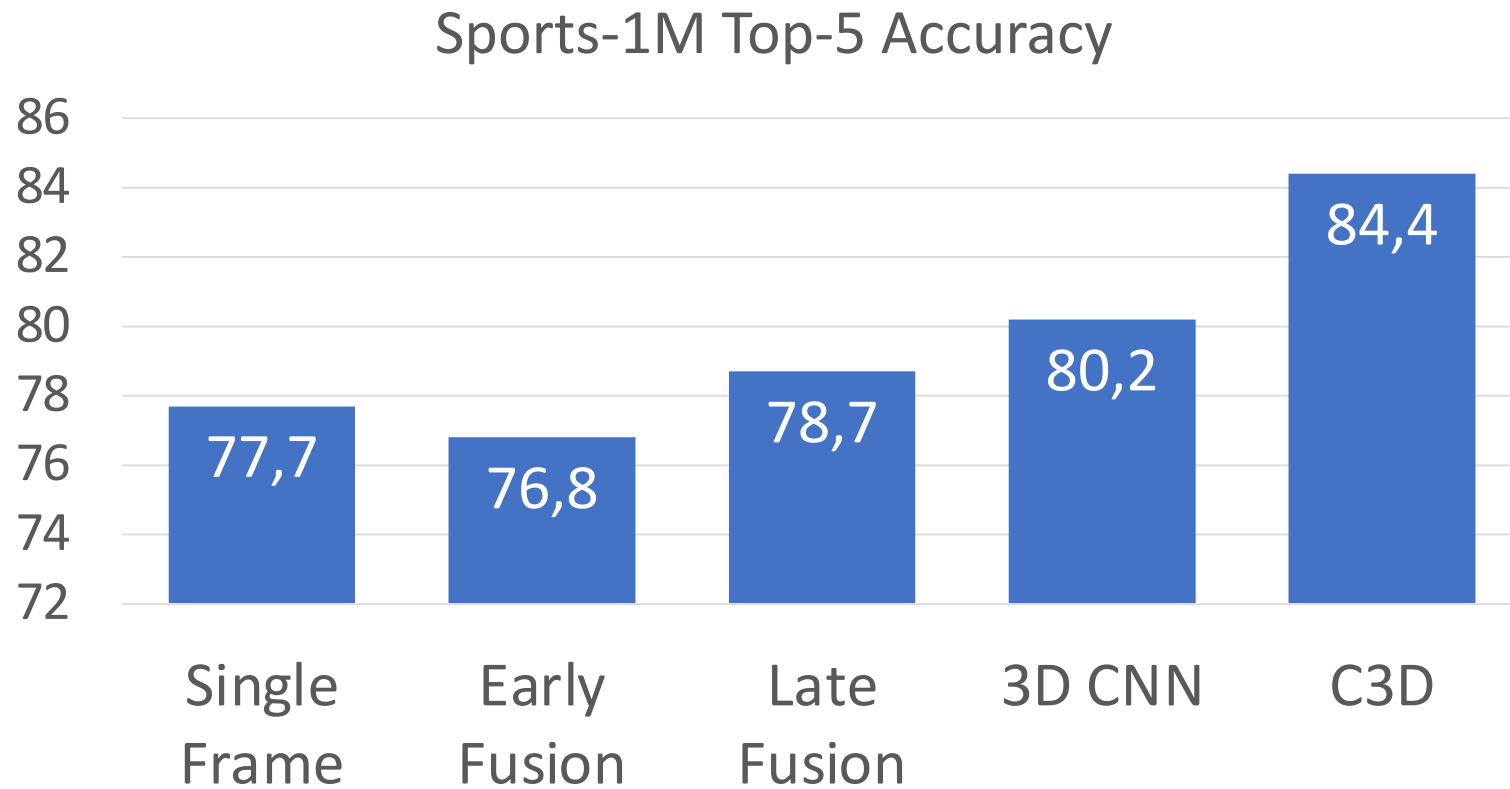
C3D: The VGG of 3D CNNs

Problem: 3x3x3 conv is very expensive!

- AlexNet: 0.7 GFLOP
- VGG-16: 13.6 GFLOP
- C3D: **39.5 GFLOP (2.9x VGG!)**

Layer	Size	MFLOPs
Input	3 x 16 x 112 x 112	
Conv1 (3x3x3)	64 x 16 x 112 x 112	1.04
Pool1 (1x2x2)	64 x 16 x 56 x 56	
Conv2 (3x3x3)	128 x 16 x 56 x 56	11.10
Pool2 (2x2x2)	128 x 8 x 28 x 28	
Conv3a (3x3x3)	256 x 8 x 28 x 28	5.55
Conv3b (3x3x3)	256 x 8 x 28 x 28	11.10
Pool3 (2x2x2)	256 x 4 x 14 x 14	
Conv4a (3x3x3)	512 x 4 x 14 x 14	2.77
Conv4b (3x3x3)	512 x 4 x 14 x 14	5.55
Pool4 (2x2x2)	512 x 2 x 7 x 7	
Conv5a (3x3x3)	512 x 2 x 7 x 7	0.69
Conv5b (3x3x3)	512 x 2 x 7 x 7	0.69
Pool5	512 x 1 x 3 x 3	
FC6	4096	0.51
FC7	4096	0.45
FC8	C	0.05

Early Fusion vs Late Fusion vs 3D CNN



Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Recognizing Actions from Motion

We can easily recognize actions using only **motion information**



Motion is very
informative
(more than
appearance?)

IDEA: capture
and process
motion

Measuring Motion: Optical Flow

Image at frame t

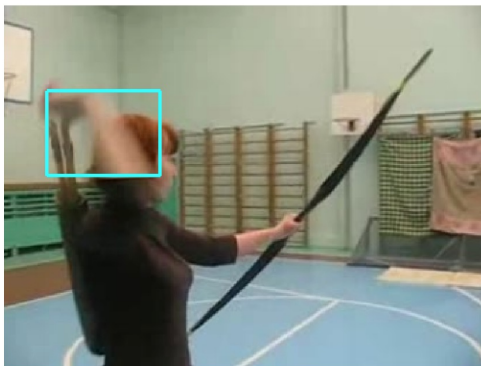
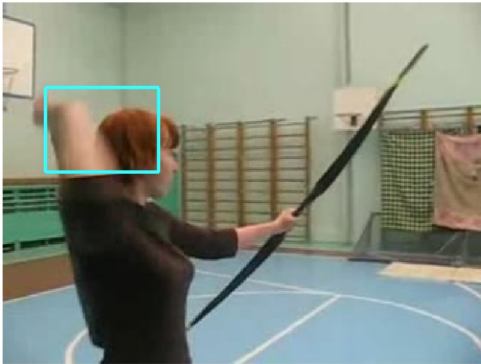


Image at frame $t+1$

Measuring Motion: Optical Flow

Image at frame t

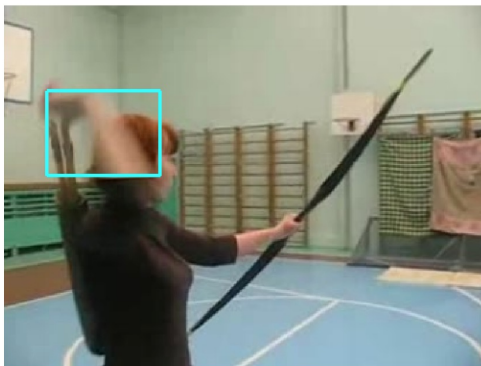
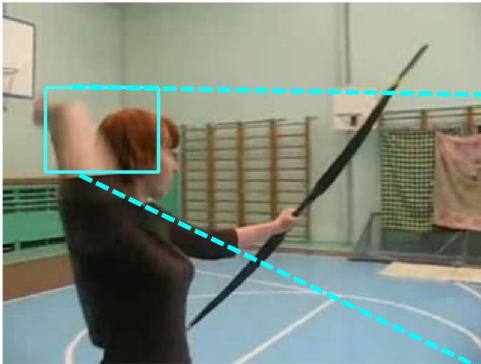
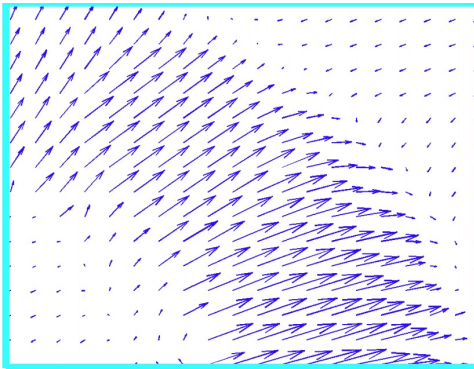


Image at frame t+1

Optical flow gives a displacement field F between images I_t and I_{t+1}



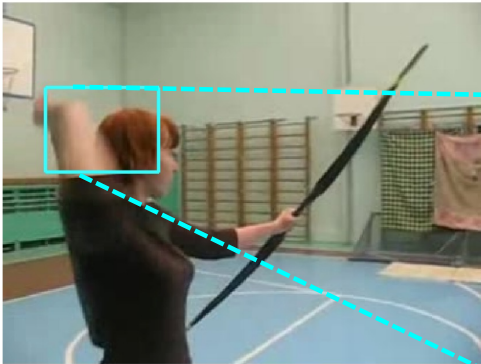
Tells where each pixel will move in the next frame:

$$F(x, y) = (dx, dy)$$

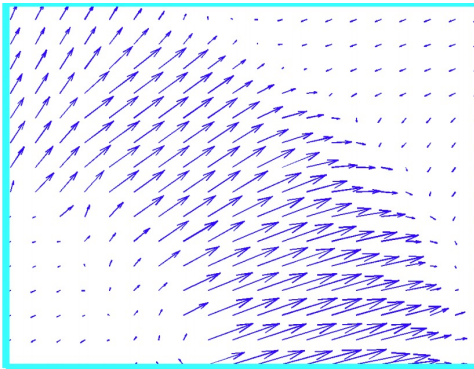
$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

Measuring Motion: Optical Flow

Image at frame t



Optical flow gives a displacement field F between images I_t and I_{t+1}

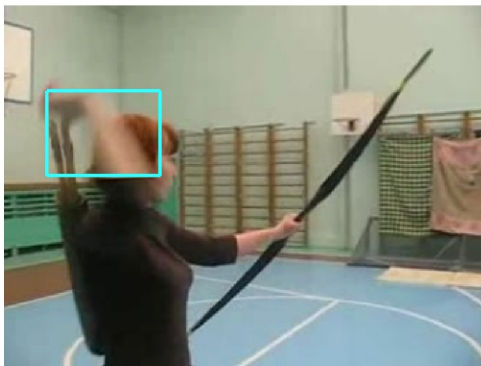


Tells where each pixel will move in the next frame:

$$F(x, y) = (dx, dy)$$

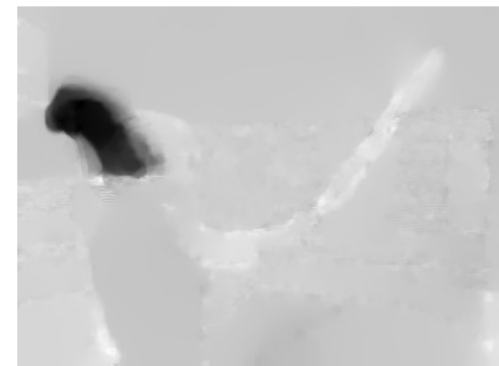
$$I_{t+1}(x+dx, y+dy) = I_t(x, y)$$

Image at frame t+1



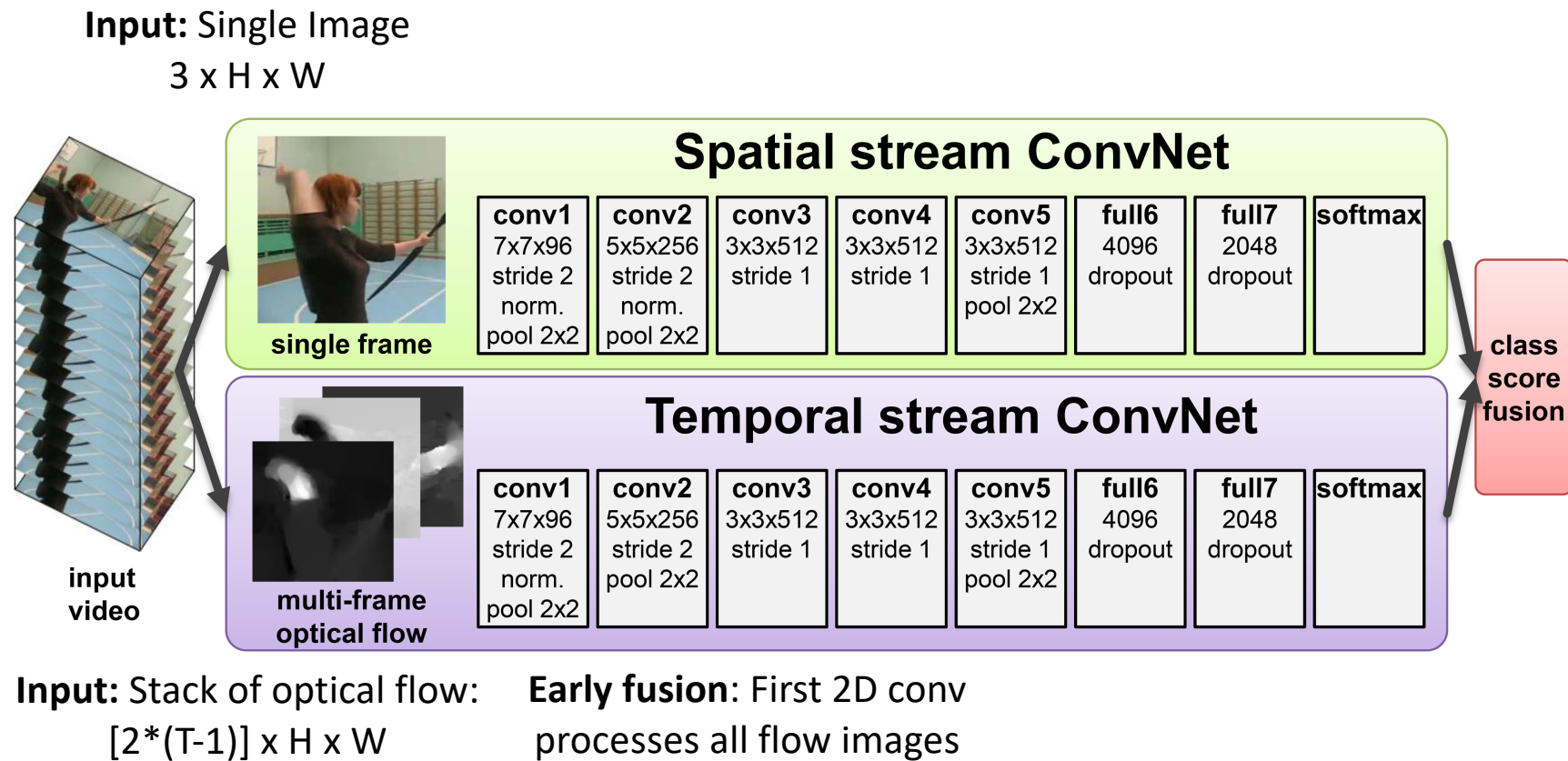
Optical Flow highlights **local motion**

Horizontal flow dx



Vertical Flow dy

Separating Motion and Appearance: Two-Stream Networks



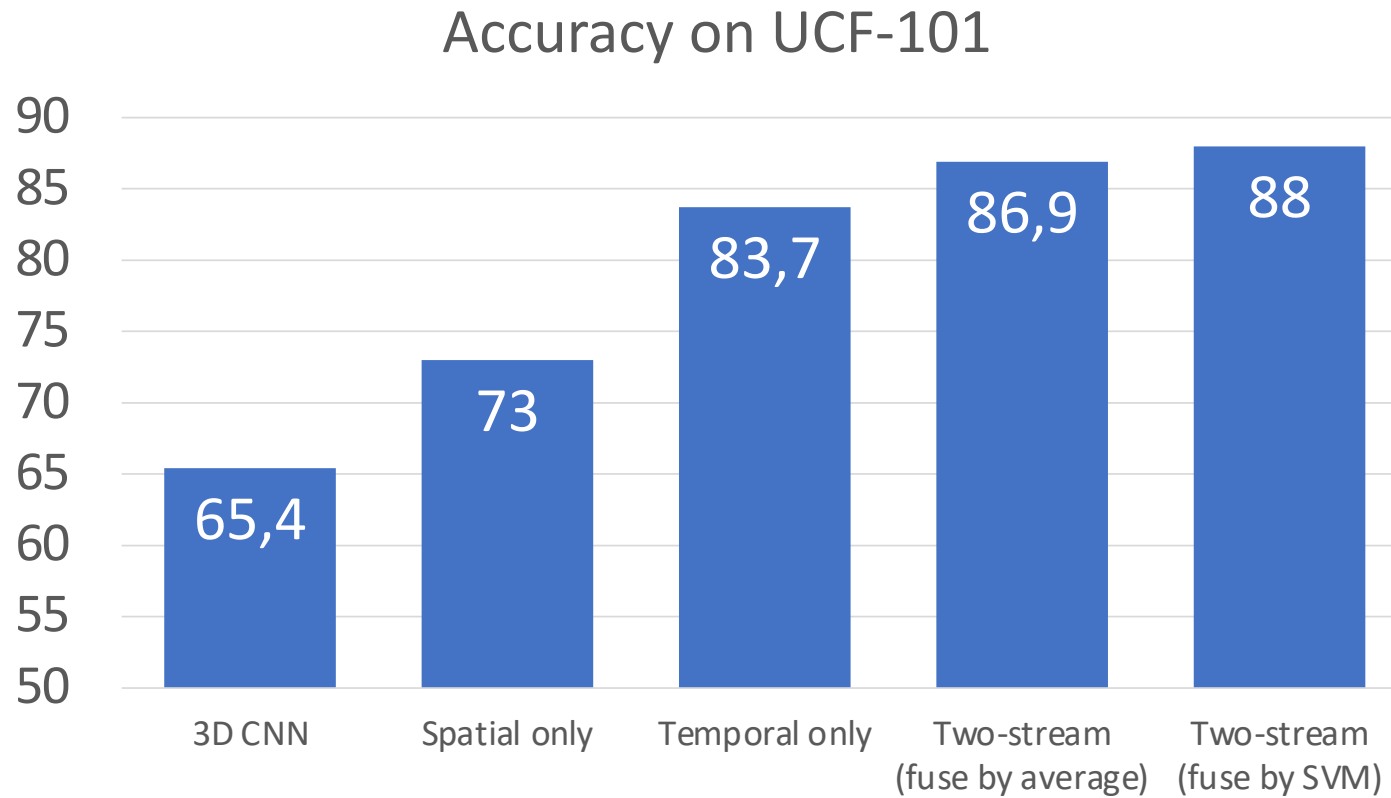
Action Dataset: UCF101

- ✓ Simple actions
- ✓ Realistic Environments
- ✓ Groups share:
 - Backgrounds
 - Actors
 - Other features

Actions	101
Clips	13320
Groups per Action	25
Clips per Group	4-7
Mean Clip Length	7.21 sec
Total Duration	1600 mins
Min Clip Length	1.06 sec
Max Clip Length	71.04 sec
Frame Rate	25 fps
Resolution	320×240
Audio	Yes (51 actions)

Soomro, K., Zamir, A.R., Shah, M.: UCF101: A dataset of 101 human actions classes from videos in the wild. CoRR abs/1212.0402 (2012)

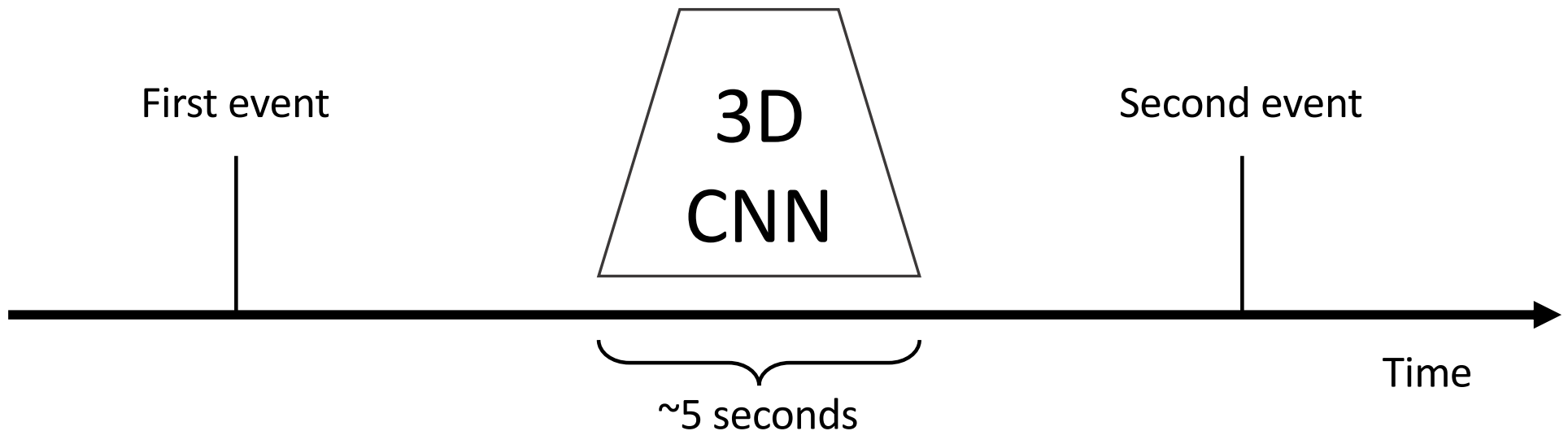
Separating Motion and Appearance: Two-Stream Networks



Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

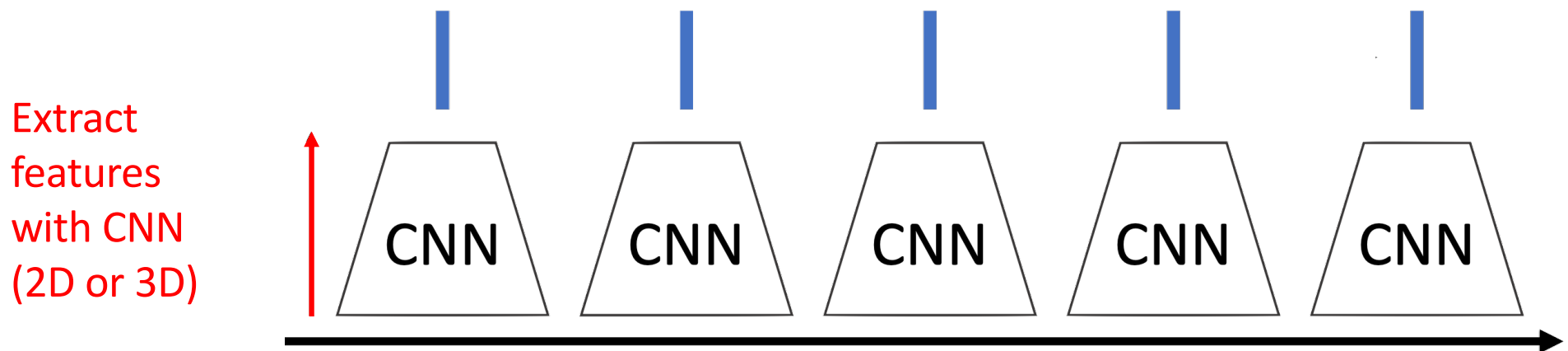
Modeling long-term temporal structure

- ✓ **Temporal CNNs** only model local motion between frames in **very short clips** of ~2-5 seconds
- ✓ What about **long-term structure**?



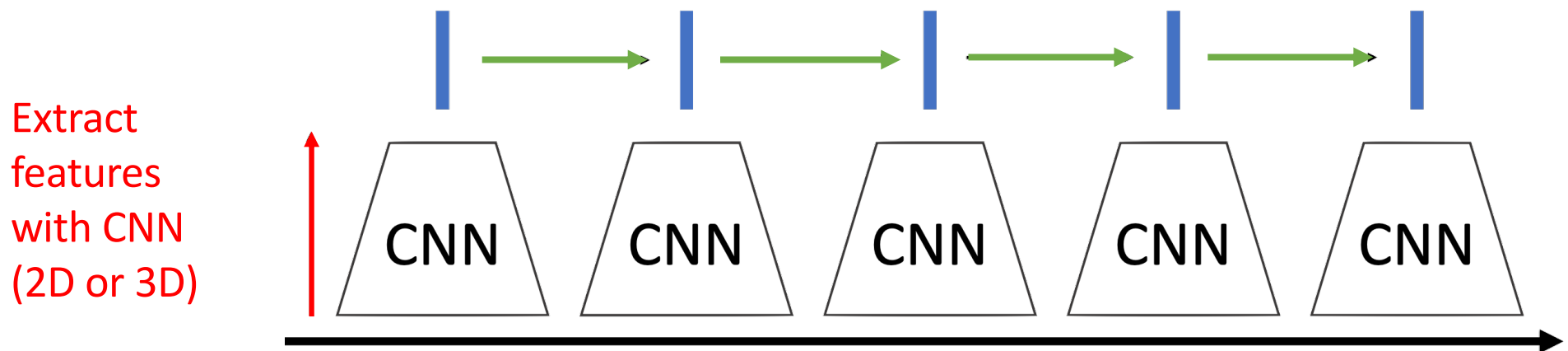
Modeling long-term temporal structure with RNN

1. Extract features with CNN



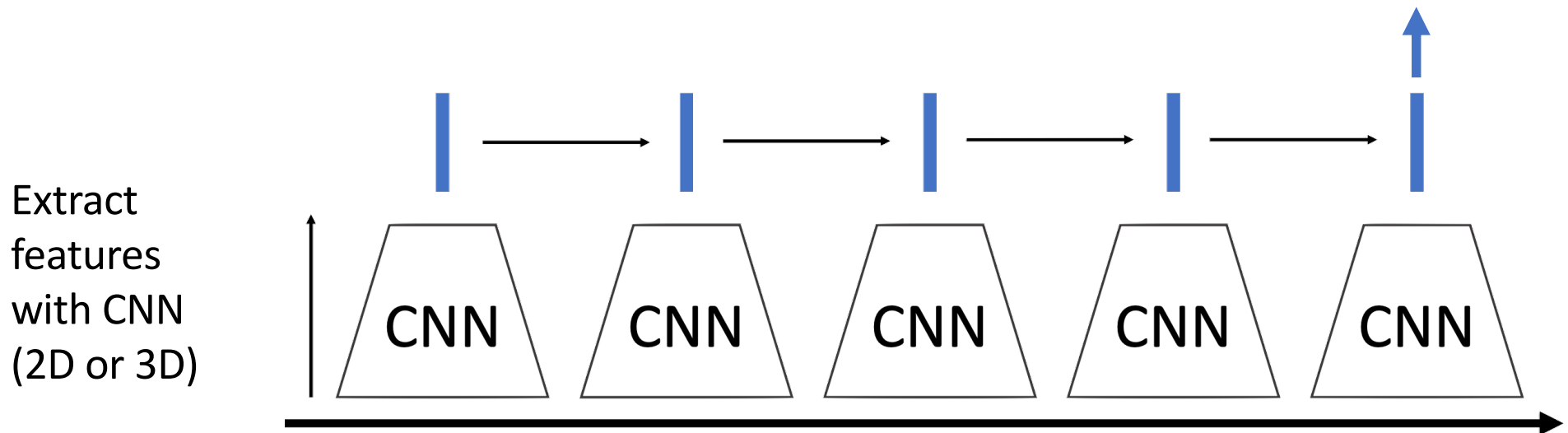
Modeling long-term temporal structure with RNN

1. Extract features with CNN
2. Process local features using recurrent network (e.g. LSTM)



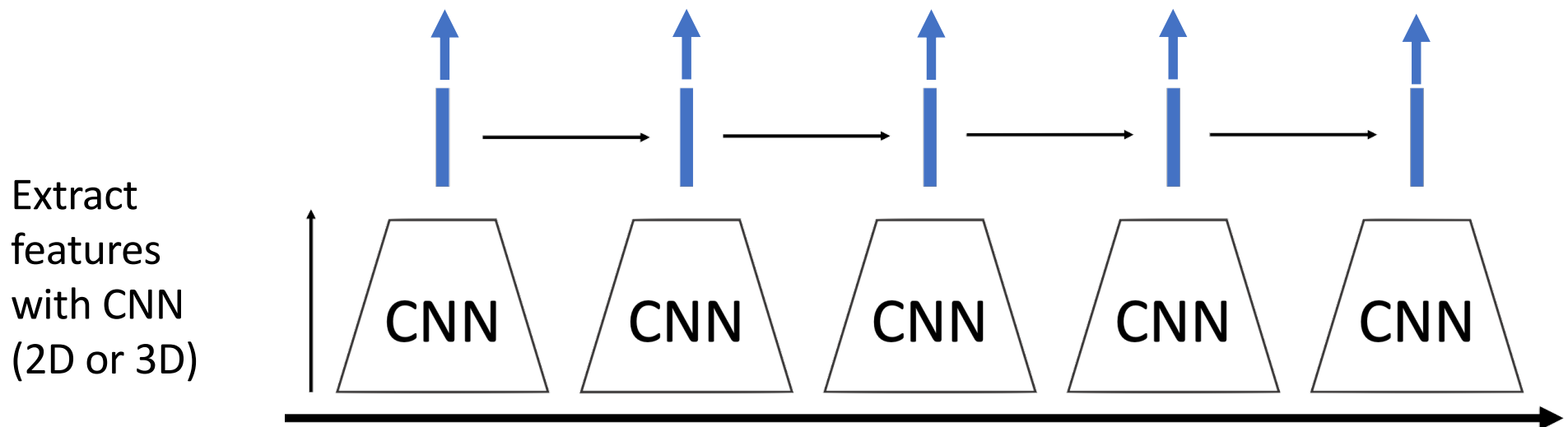
Modeling long-term temporal structure with RNN

- ✓ **Process local features** using **recurrent network** (e.g. LSTM)
- ✓ **Many to one**: One output at end of video



Modeling long-term temporal structure with RNN

- ✓ **Process local features** using **recurrent network** (e.g. LSTM)
- ✓ **Many to many**: one output per video frame

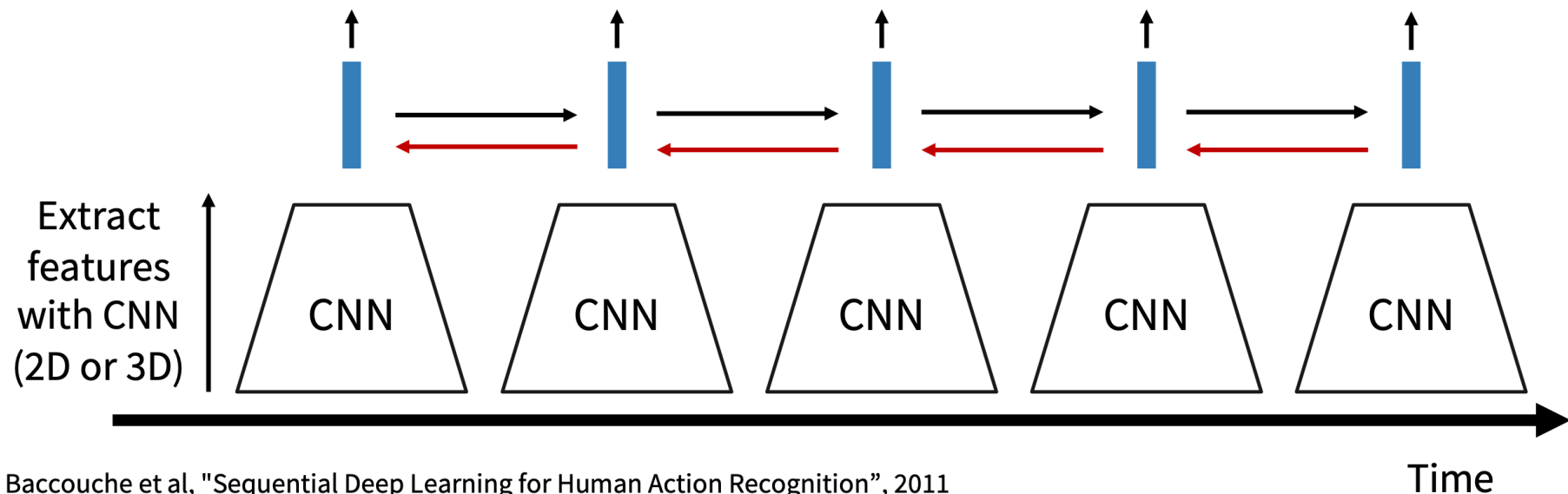


Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011

Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Modeling long-term temporal structure with RNN

Sometimes don't backprop to CNN to save memory:
pretrain and use CNN as a feature extractor



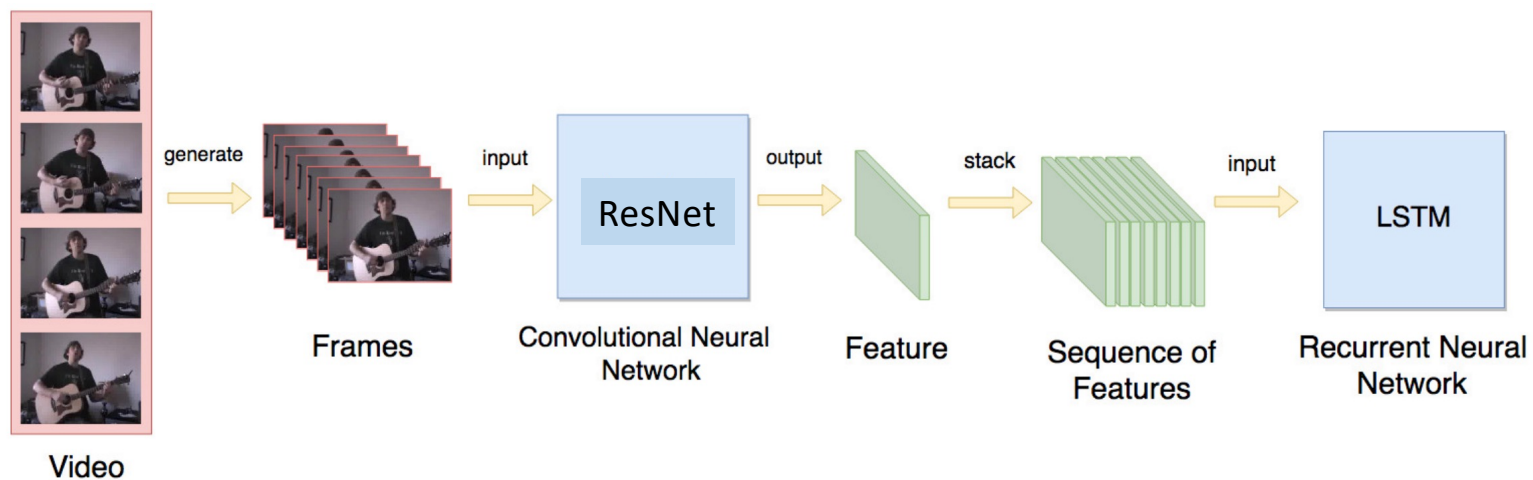
Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011

Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Slide credit: Justin Johnson

Design choice

- ✓ The **final layers** of a **CNN** contain important **properties** related to a **frame**
- ✓ Represented as an **array** of values **given** by a **prediction** for each RGB **frame** for the video clips
- ✓ **LSTM layers** are composed by units which consider what previously happens and a **memory state** to generate a prediction
- ✓ The **output** of the **LSTM** layer is passed to **Dense** layers followed by a final **softmax** layer



Demo

(Inspired by: <https://github.com/eriklindernoren/Action-Recognition>)

- ✓ **Download** the code folder Action-Recognition-master from my github (*.py original files, *.ipynb rearranged notebooks)
- ✓ **Download** the dataset using my `download_ucf101.sh`:

```
$ cd data/  
$ bash download_ucf101.sh  
$ unrar x UCF101.rar  
$ unzip ucfTrainTestlist.zip
```

- ✓ Before extracting frames, **remove** the corrupted videos:
 - `v_Surfing_g07_c02.avi`
 - `v_Surfing_g02_c03.avi`
- ✓ **Extract** frames: `$ python3 extract_frames.py`

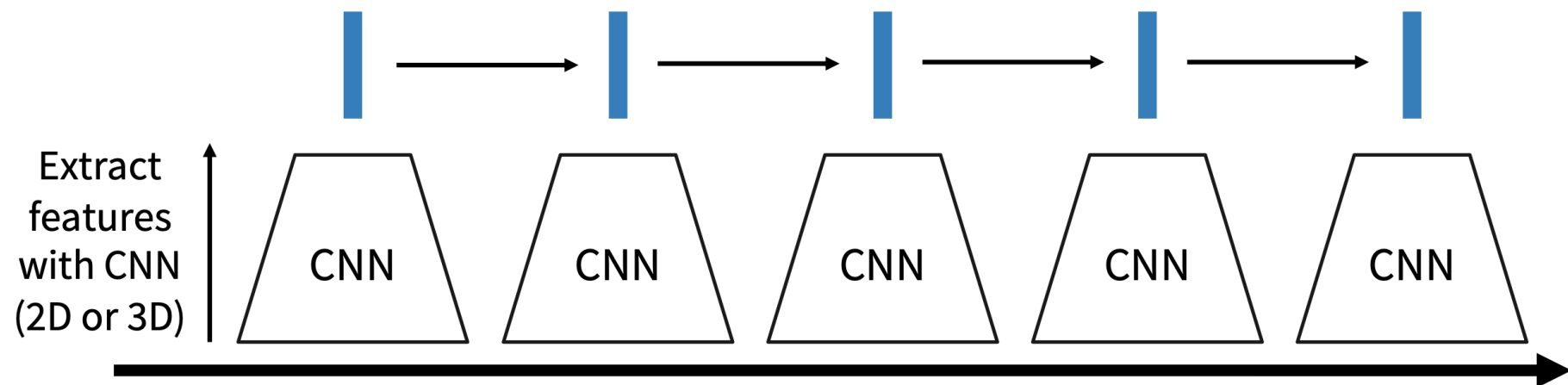
Demo

- ✓ Download the model ConvLSTN_90.pth from: [repository](#)
- ✓ Save it in the folder `model_checkpoints`
- ✓ Now you can **run** the `test` or the `test_on_video`
- ✓ You can also **learn** new models running the notebook `train`
- ✓ **Inspect** the notebook `model`

Modeling long-term temporal structure with RNN

- **CNN**: function of a fixed temporal window (local temporal structure)
- **RNN**: function of all previous vectors (global temporal structure)

Can we merge both approaches?



Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011

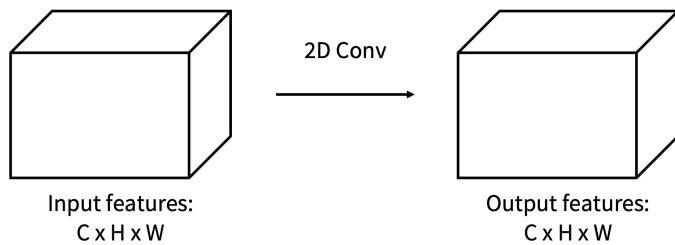
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Time

Slide credit: Justin Johnson

Recurrent Convolutional Network

- Recall CNN:



- Recall RNN:

e.g. 3-layer RNN

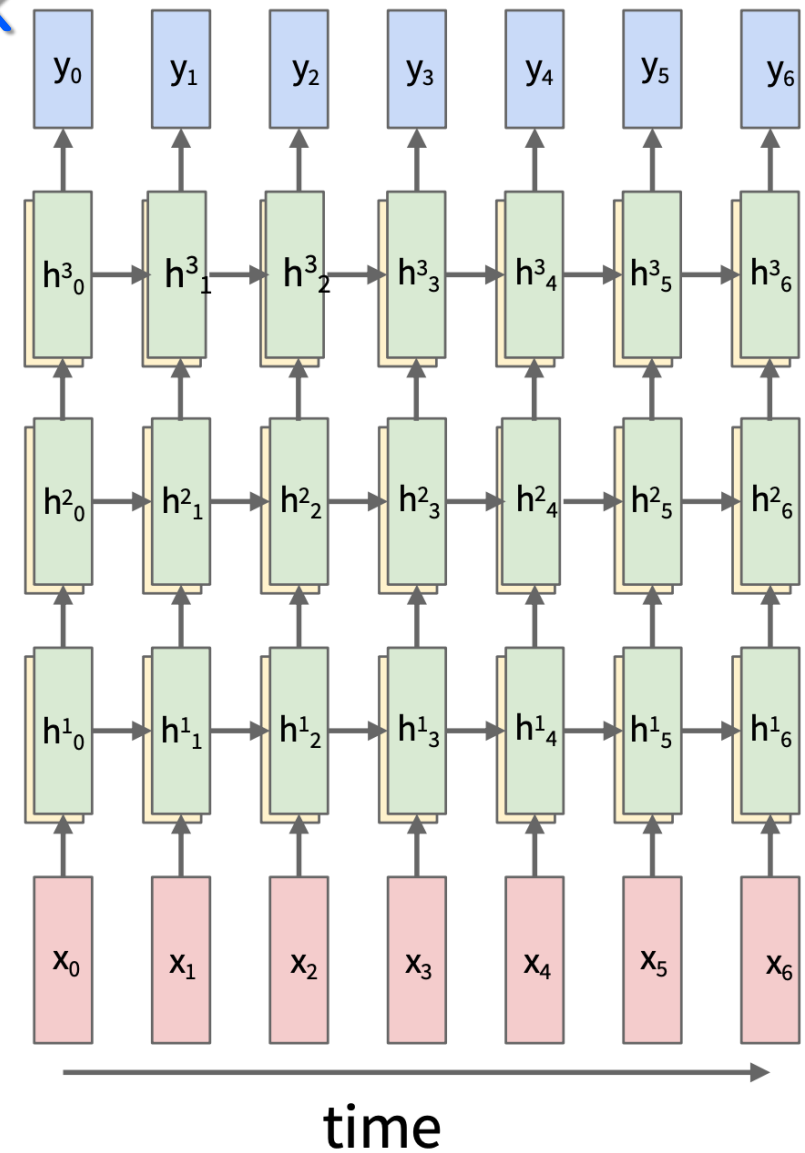
$$h_t = f_W(h_{t-1}, x_t)$$

new state

old state

some function
with parameters W

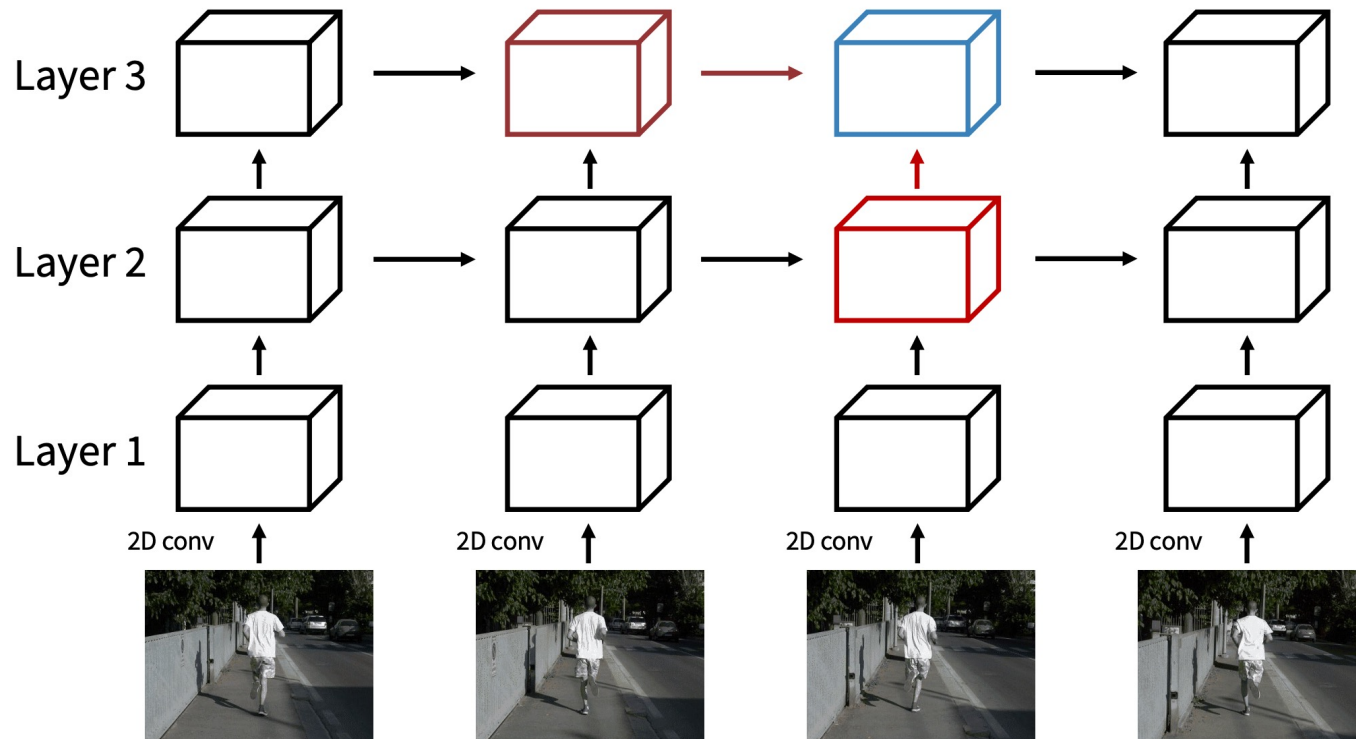
depth



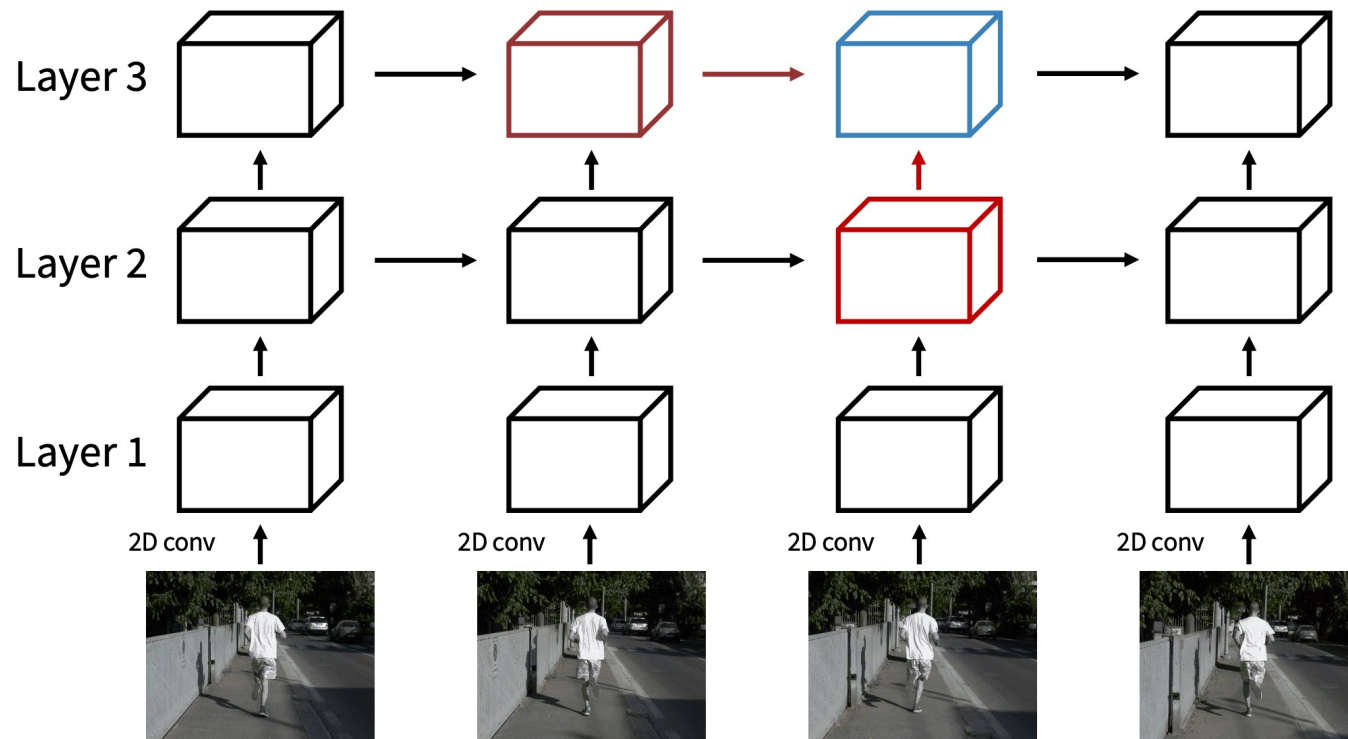
Recurrent Convolutional Network

- Can we merge both approaches?

- Multi-layer RNN
- Maintaining the spatial structure



Recurrent Convolutional Network



Entire network
uses 2D feature
maps: $C \times H \times W$

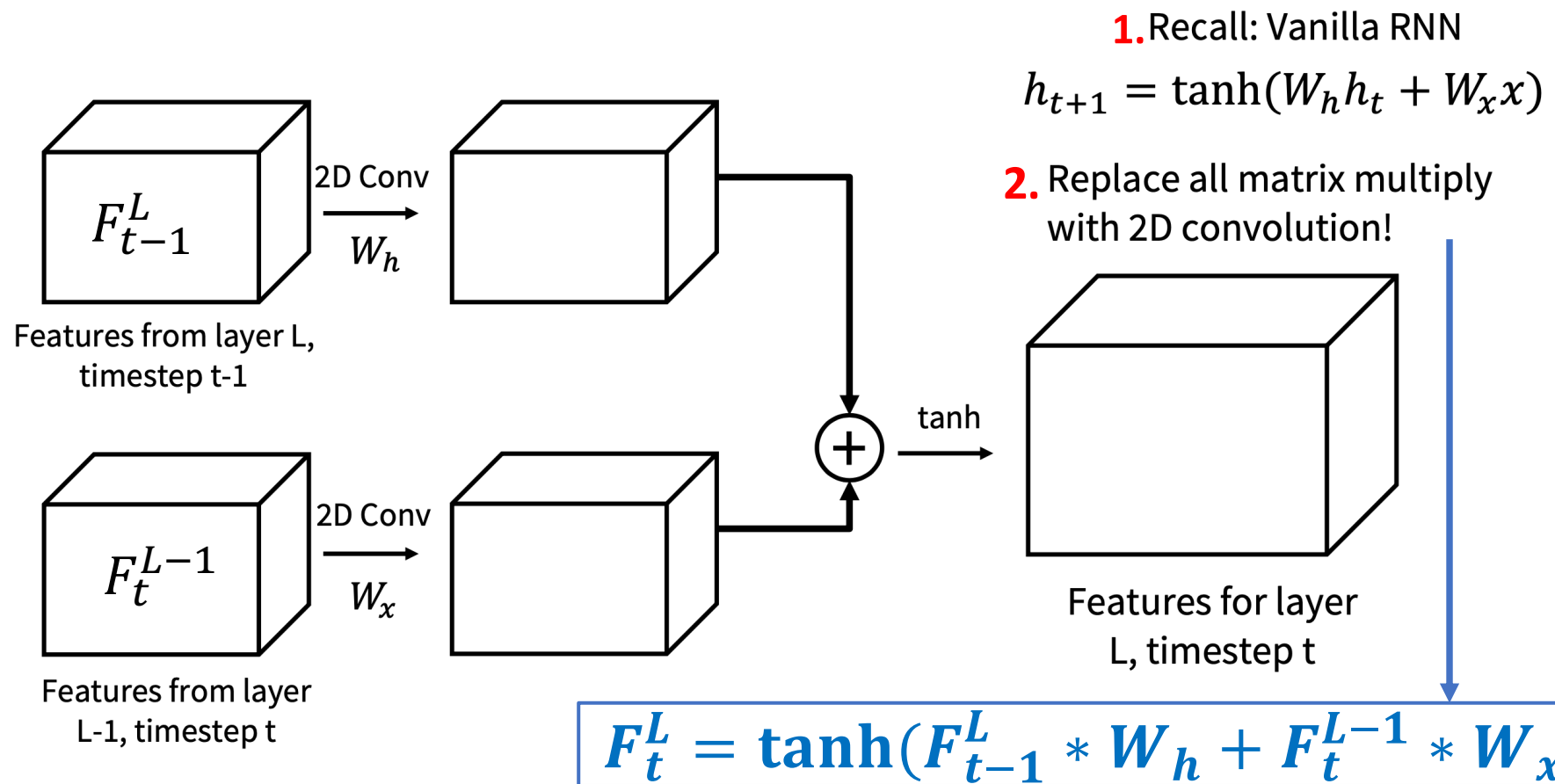
Each depends on
two inputs:

1. Same layer,
previous timestep
2. Prev layer,
same timestep

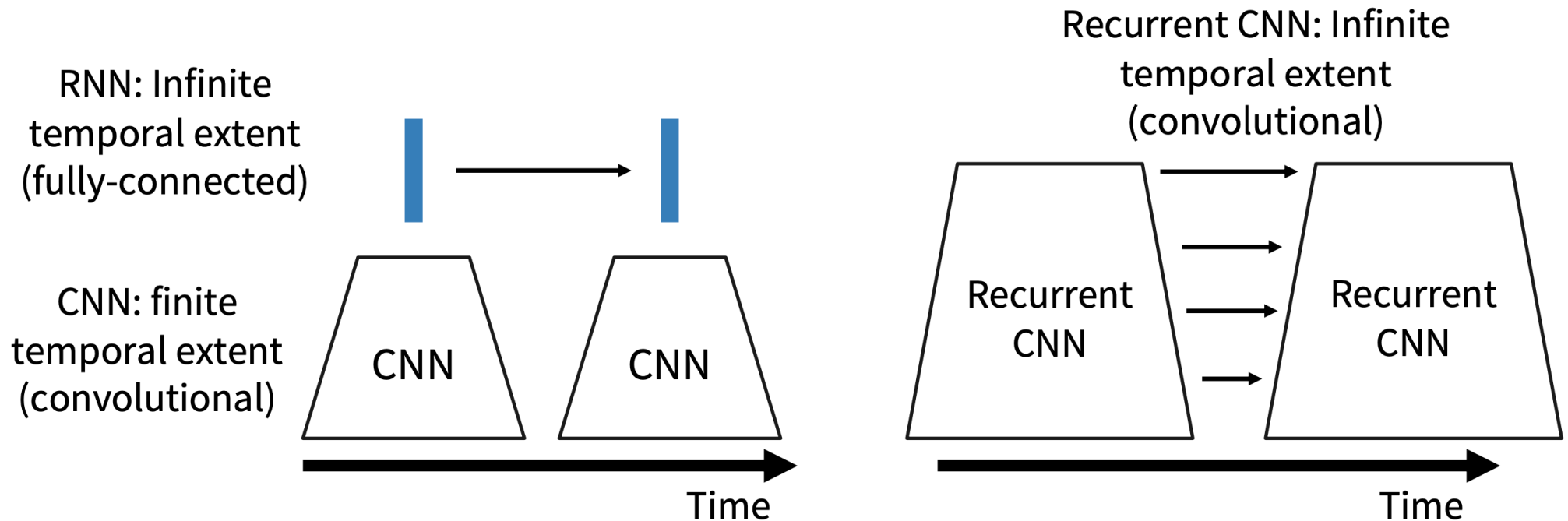
Use different weights
at each layer, share
weights across time

Ballas et al, "Delving Deeper into Convolutional Networks for Learning Video Representations", ICLR 2016

Recurrent Convolutional Network



Modeling long-term temporal structure



Inflating 2D Networks to 3D (I3D)

There has been a lot of work on architectures for images.

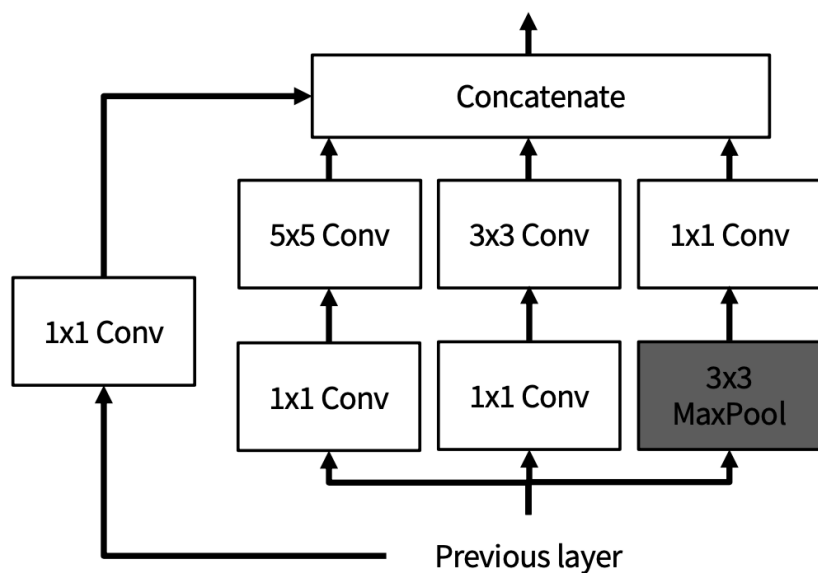
- Can we reuse image architectures for video?

Idea: take a 2D CNN architecture.

Replace each 2D $K_h \times K_w$ conv/pool layer
with a 3D $K_t \times K_h \times K_w$ version

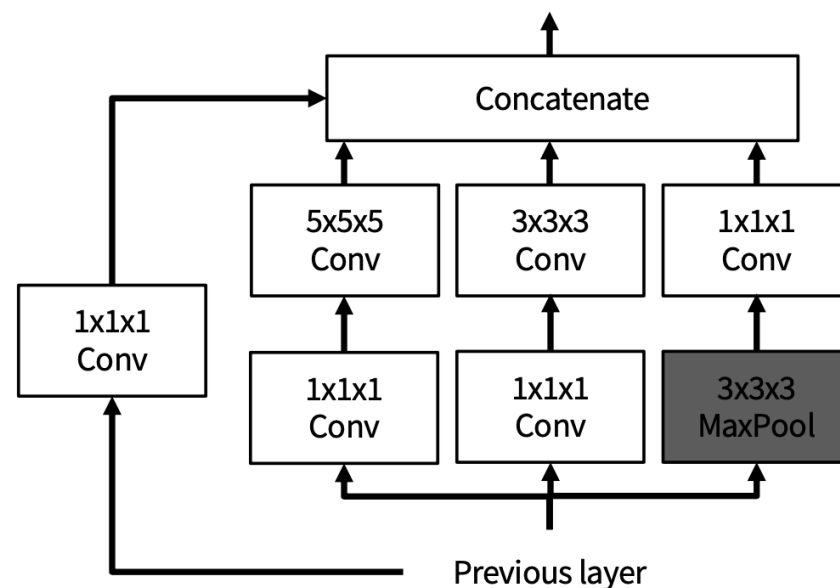
Inflating 2D Networks to 3D (I3D)

Inception Block: Original



2D $K_h \times K_w$ conv/pool layer

Inception Block: Inflated



3D $K_t \times K_h \times K_w$ conv/pool layer

Inflating 2D Networks to 3D (I3D)

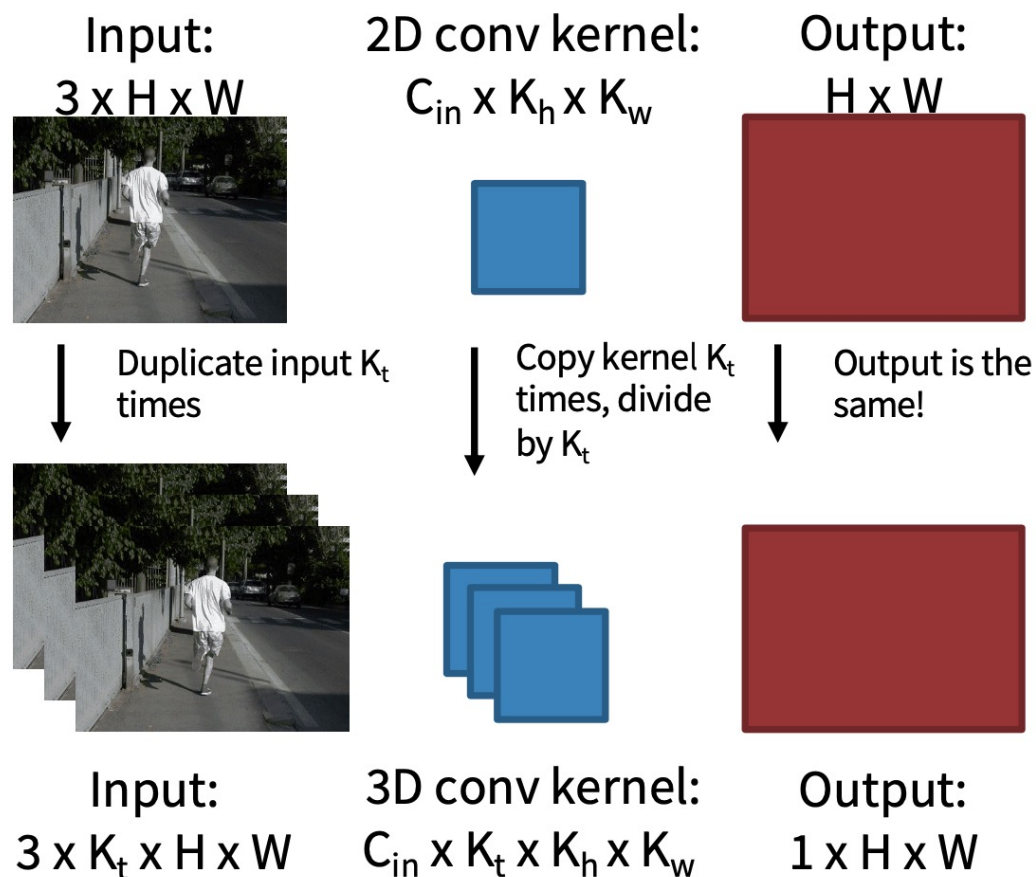
IDEA for REUSE: use weights of 2D conv to initialize 3D conv:



Copy K_t times the spatial kernel and divide by K_t

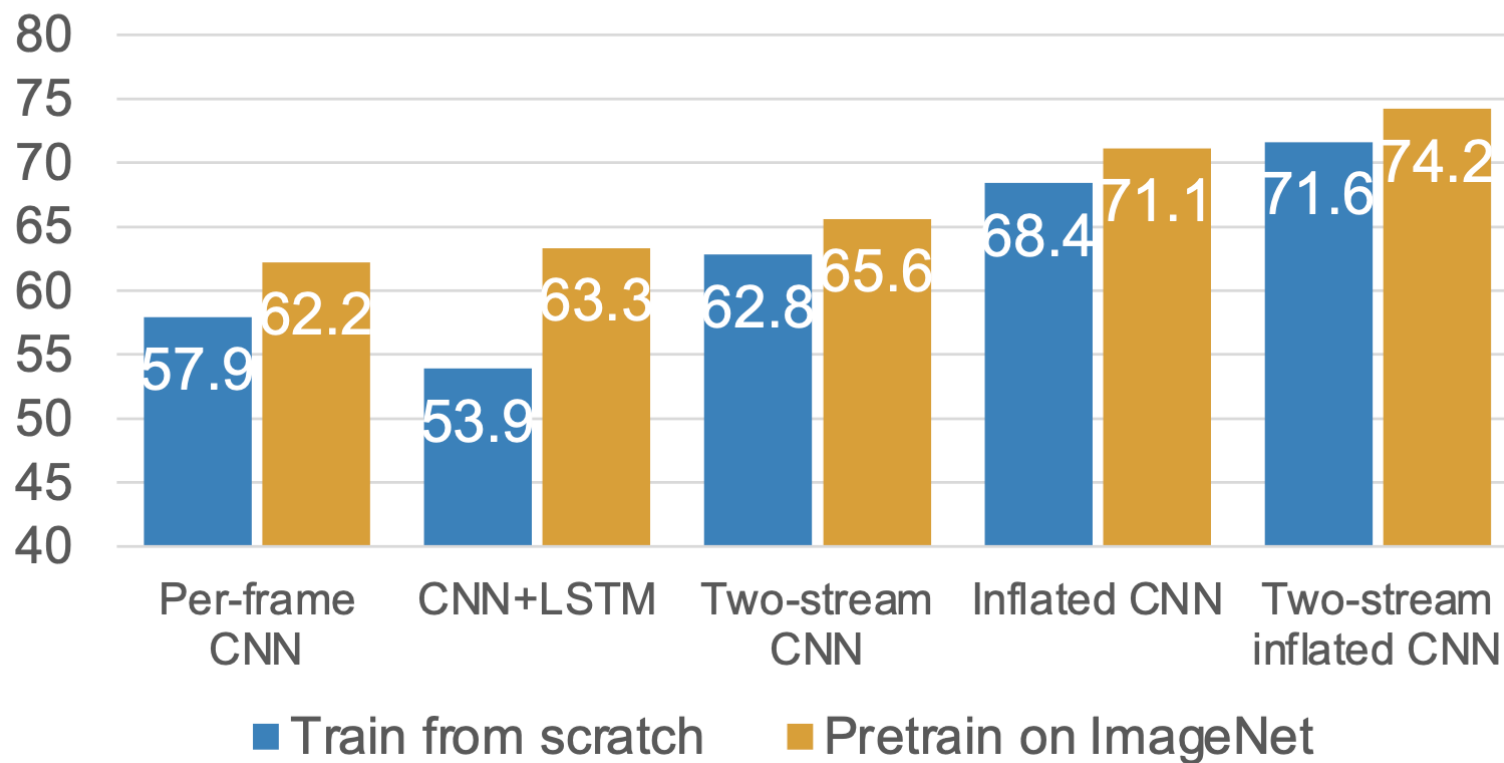


This gives the same result as 2D conv given “constant” video input



Inflating 2D Networks to 3D (I3D)

Top-1 Accuracy on Kinetics-400

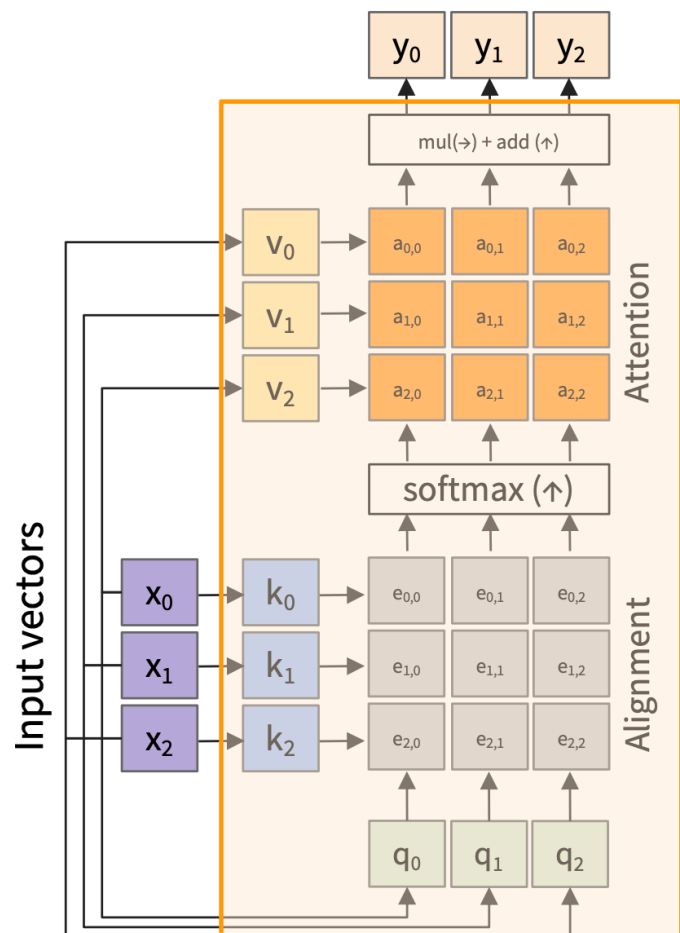


All using Inception CNN

Beyond Recurrence

- Self-Attention and Transformers (**Next Lecture**)

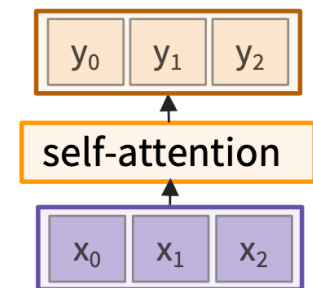
Recall Self-Attention



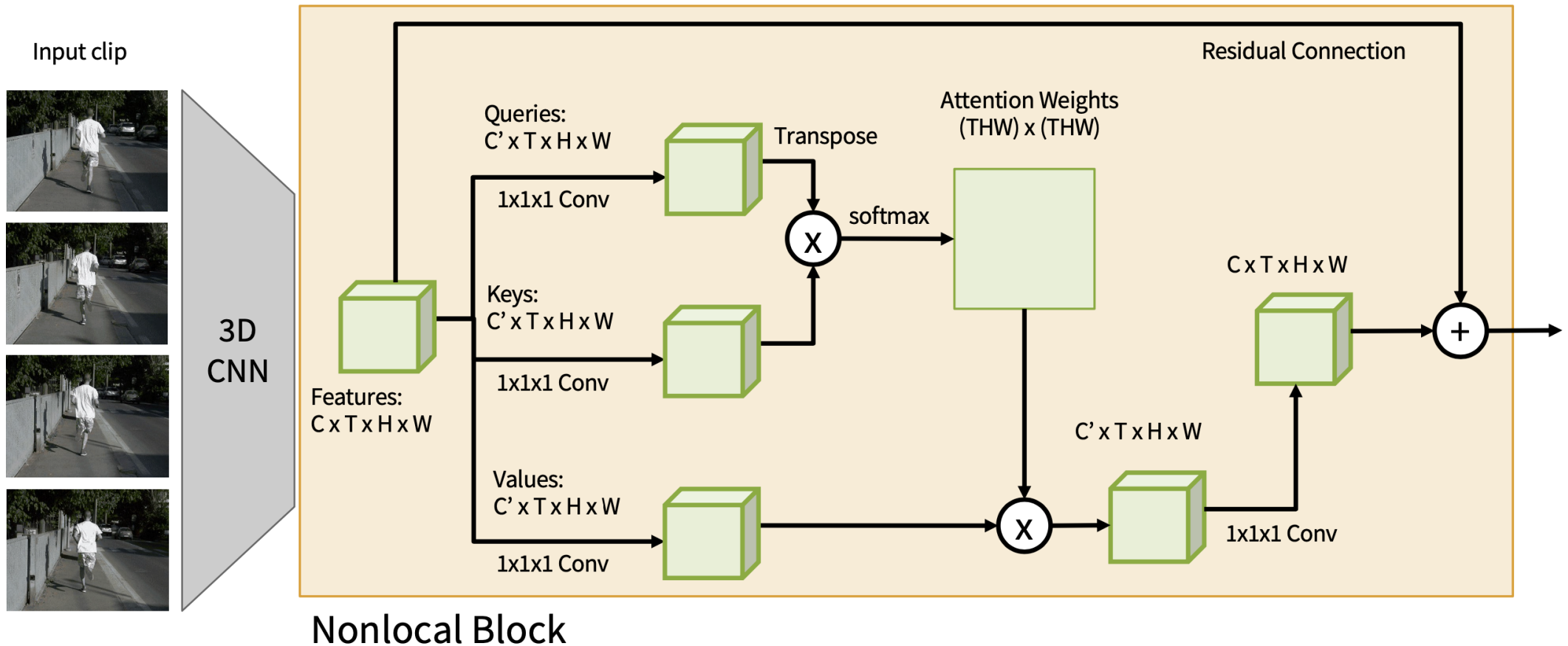
Outputs:
context vectors: y (shape: D_v)

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: x (shape: $N \times D$)

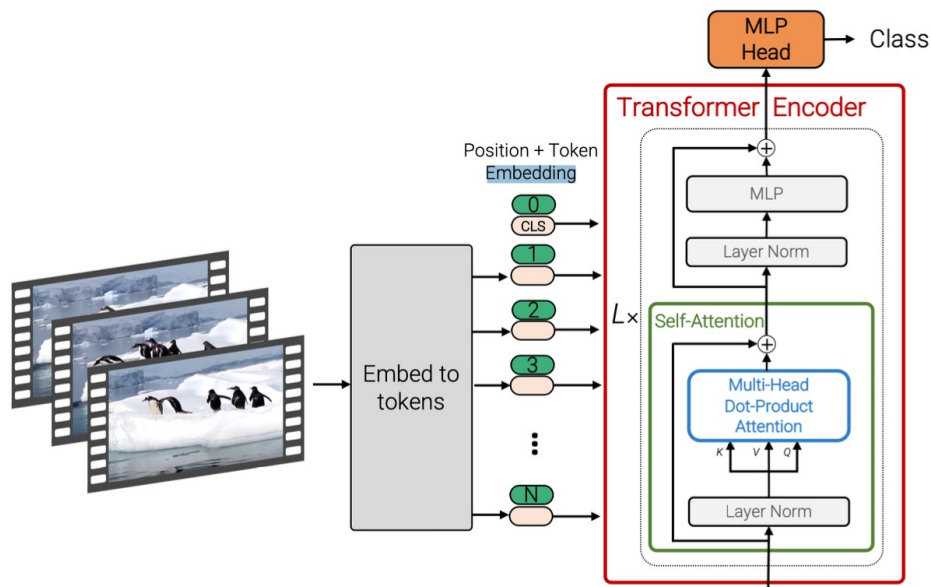


Spatio-temporal Self-Attention (Non-Local Block)



Vision Transformers for Videos

Factorized attention: Attend over space / time

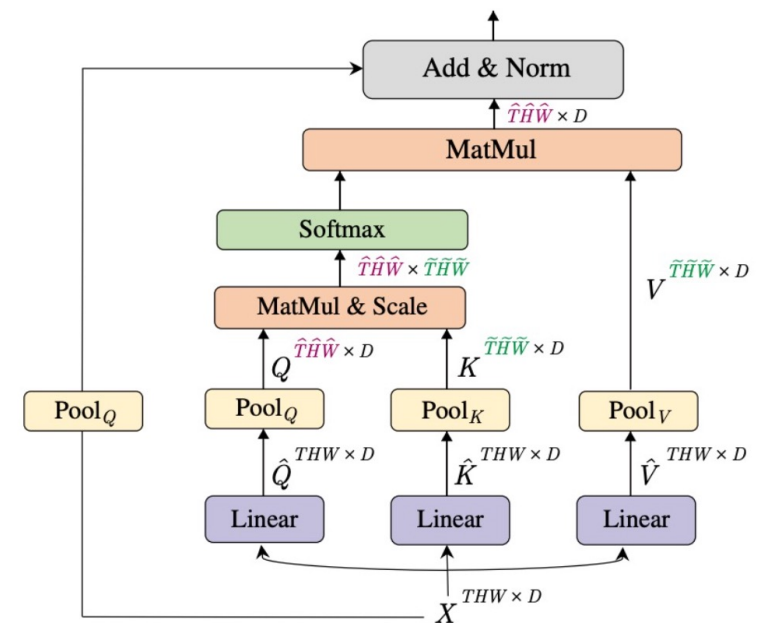


Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021

Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021

Neimark et al, "Video Transformer Network", ICCV 2021

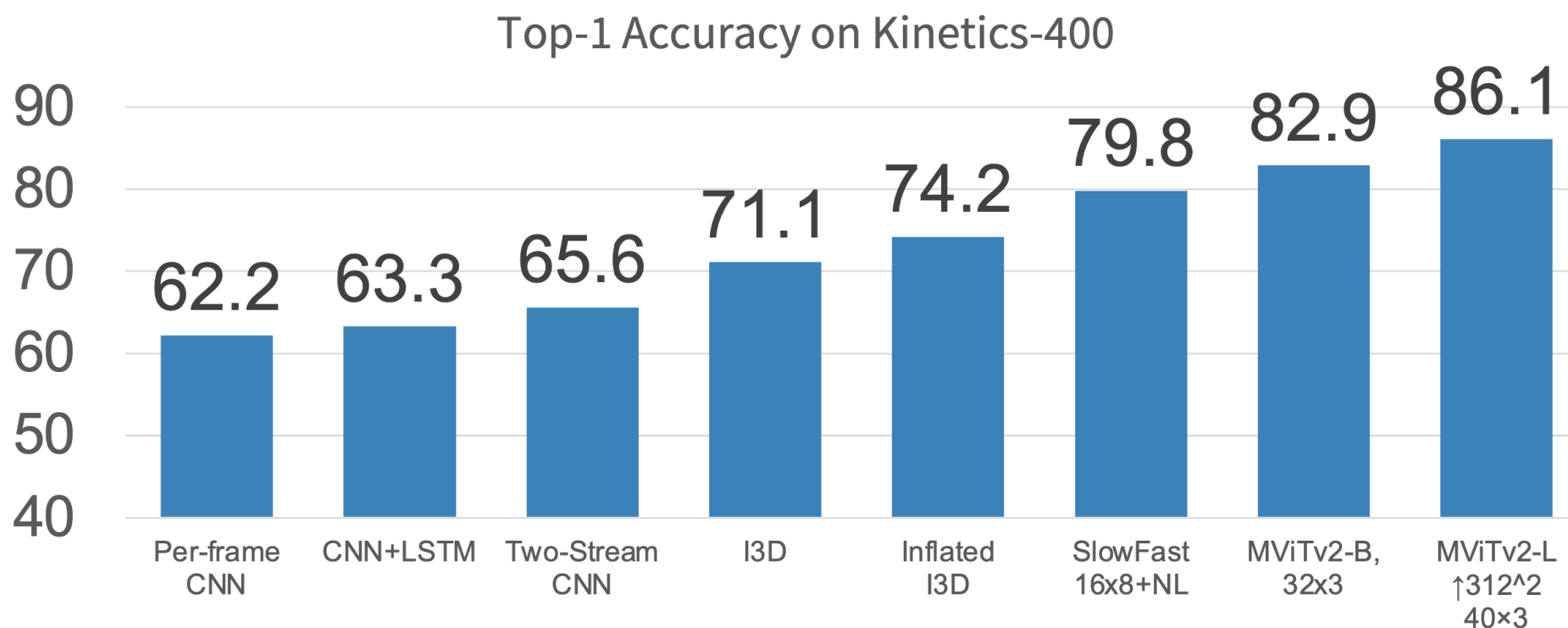
Pooling module: Reduce number of tokens



Fan et al, "Multiscale Vision Transformers", ICCV 2021

Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

Vision Transformers for Videos



Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

Attention mechanism & Transformers

On the notes...

Vision Transformers (ViTs)

